CENTRE FOR DISTANCE AND ONLINE EDUCATION
SCHOOL OF DISTANCE EDUCATION
பாரதியார் பல்கலைக்கழகம்
BHARATHIAR UNIVERSITY
COIMBATORE – 641 046, TAMILNADU, INDIA
| State University | Category-I University by UGC | A++ Grade - 3.63 CGPA by NAAC | 26th Rank among Indian Universities by MoE- NIRF

Title of the Project Report:

TASKFLOW – INTELLIGENT TASK ALLOCATION AND TRACKING SYSTEM

The Project Report submitted to Bharathiar University in partial fulfillment of the Requirement for the award of the Degree of Master of Computer Applications

JOJO GEORGE

Enrolment No. : <Enrolment No.>

Reg. No. : 251MCAN0024

Under the guidance of

<NAME OF THE GUIDE>

<Designation>

Centre for Distance and Online Education
Bharathiar University
Coimbatore 641 046

DECEMBER 2026

CENTRE FOR DISTANCE AND ONLINE EDUCATION
SCHOOL OF DISTANCE EDUCATION
பாரதியார் பல்கலைக்கழகம்
BHARATHIAR UNIVERSITY
COIMBATORE – 641 046, TAMILNADU, INDIA
| State University | Category-I University by UGC | A++ Grade - 3.63 CGPA by NAAC | 26th Rank among Indian Universities by MoE- NIRF

ANNEXURE-I

DECLARATION

I hereby declare that this project work titled **TASKFLOW – INTELLIGENT TASK ALLOCATION AND TRACKING SYSTEM** submitted to the Centre for Distance and Online Education, Bharathiar University is a record of original work done by **JOJO GEORGE** under the supervision and guidance of **<NAME OF THE GUIDE>** and that this work has not formed the basis for the award of any Degree/Diploma/Associate ship/Fellowship or similar title to any candidate of any University.

Signature of the candidate

Name : JOJO GEORGE

Enrolment No. : <Enrolment No.>

Register No : 251MCAN0024

Course : Master of Computer Applications (MCA)

Place : <Place>

Date :

Countersigned by

Signature of the Guide
(With Seal)

Signature of the Co-ordinator
(With seal)

CENTRE FOR DISTANCE AND ONLINE EDUCATION

SCHOOL OF DISTANCE EDUCATION

பாரதியார் பல்கலைக்கழகம்

BHARATHIAR UNIVERSITY

COIMBATORE – 641 046, TAMILNADU, INDIA

| State University | Category-I University by UGC | A^{++} Grade - 3.63 CGPA by NAAC | 26th Rank among Indian Universities by MoE- NIRF

ANNEXURE – II

CERTIFICATE

This is to certify that the project work titled **TASKFLOW – INTELLIGENT TASK ALLOCATION AND TRACKING SYSTEM** submitted to Bharathiar University in partial fulfillment of the requirements for the award of the Degree of Master of Computer Applications is a record of the original work done by **JOJO GEORGE** under my supervision and guidance and that this project work has not formed the basis for the award of any Degree/Diploma/Associate ship/Fellowship or similar title to any candidate of any University.

Signature of the Guide
(With Seal)

Programme Co-ordinator
(With Seal)

Forwarded by

Director
Centre for Distance and Online Education
Bharathiar University
Coimbatore – 46

Annexure – III

CONTENTS

TITLE **PAGE NO.**

ACKNOWLEDGEMENT ..... **i**
SYNOPSIS ..... **ii**
1. INTRODUCTION **1**
1.1 SYSTEM OVERVIEW 1
1.2 ORGANIZATION PROFILE 2
2. SYSTEM STUDY AND ANALYSIS **4**
2.1 PROBLEM STATEMENT 4
2.2 EXISTING SYSTEM 5
2.2.1. Drawbacks 5
2.3 PROPOSED SYSTEM 6
2.3.1. Advantages 6
2.4 FEASIBILITY ANALYSIS 7
2.4.1. Technical Feasibility 7
2.4.2. Economic Feasibility 8
2.4.3. Operational Feasibility 8
2.4.4. Cost Estimation And Scheduling 8
3. DEVELOPMENT ENVIRONMENT **11**
3.1 HARDWARE REQUIREMENT 11
3.2 SOFTWARE REQUIREMENT 12
3.3 PROGRAMMING ENVIRONMENT 13
3.3.1. About ASP.NET Core And C# 13
3.3.2. About Angular And TypeScript 14
3.3.3. About SQLite And Entity Framework Core 15
4. SYSTEM DESIGN AND DEVELOPMENT **16**
4.1 ELEMENTS OF DESIGN 16
4.1.1. Process Design 16
4.1.2. Concept Design 29
4.1.3. Logical Design 31
4.1.4. Physical Design 33
4.1.5. Input Design 34
4.1.6. Output Design 43
4.1.7. Database Design 55
4.2 TABLE STRUCTURE 58
5. SYSTEM TESTING AND IMPLEMENTATION **72**
5.1 SYSTEM TESTING 72
5.1.1. Unit Testing 73

5.1.2. Integration Testing....73
5.1.3. User Acceptance Testing....74
5.1.4. Output Testing....75
5.1.5. Validation Testing....76
5.2 SYSTEM SECURITY....76
5.3 SYSTEM IMPLEMENTATION....77
5.4 SYSTEM ENHANCEMENT....78
6. CONCLUSION AND FURTHER ENHANCEMENT....80
6.1 CONCLUSION....80
6.2 FURTHER ENHANCEMENT....81
7. BIBLIOGRAPHY AND REFERENCES....82
7.1 BOOKS....82
7.2 WEB REFERENCES....82

ACKNOWLEDGEMENT

The successful completion of this mini project has been made possible through the guidance, encouragement and support of many people, and a deep sense of gratitude has been felt by the candidate towards each one of them. Sincere thanks have been expressed to the Director, Centre for Distance and Online Education, Bharathiar University, Coimbatore, for the opportunity and the academic framework within which this project has been carried out as a part of the Master of Computer Applications programme. The facilities, the study material and the clear project guidelines that have been issued by the Centre have been of great help throughout the analysis, design, development and documentation stages of the work.

The candidate has been especially grateful to the project guide, <Name of the Guide>, <Designation>, for the valuable suggestions, timely reviews and constant encouragement that have been given at every stage of the project. The comments that have been received during the reviews have helped in shaping the scope of TaskFlow, in improving the structure of the design documents and in presenting this report in the format that has been prescribed by the University. Gratitude has also been extended to the Programme Co-ordinator and to all the faculty members of the Centre, whose lectures on software engineering, database management systems and web technologies have formed the foundation on which this work has been built.

Heartfelt thanks have been expressed to the family members and friends of the candidate, who have patiently supported the long hours that have been spent on coding, testing and documentation, and who have also helped by trying out the screens of the application as ordinary users. The feedback that has been given by them on the dashboards, the reminders and the task screens has been taken into account while the user interface was being refined. Finally, the open source communities behind .NET, ASP.NET Core, Entity Framework Core, SQLite, Angular, Chart.js and the many other libraries and tools that have been used have been gratefully acknowledged, since a system of this size could not have been completed within the available time without their work.

JOJO GEORGE

SYNOPSIS

TaskFlow – Intelligent Task Allocation and Tracking System has been developed as a web-based office task management application in order to simplify the way in which work is allocated to employees, tracked until completion and reported to the management of an organisation. In many small and medium organisations, tasks have traditionally been handed over through word of mouth, chat messages, spreadsheets and e-mail, and as a result no single reliable list of who was doing what and by when has been available. Deadlines have been remembered or forgotten by individuals, workload has remained invisible, and late work has been noticed by managers only after it had already become late, and TaskFlow has been designed to address these problems through one secure, role-based application.

In the system that has been developed, a task is created by a manager with a title, project, assignee, priority, start date, deadline, estimated hours, additional notification recipients and reminder intervals, and the task is saved with a unique code in the To Do status. A fixed status life cycle of To Do, In Progress, In QA, Blocked, Completed and Cancelled has been enforced for every task, the hours that have been worked are logged by the assignee, and the work is discussed through task comments while every important change is recorded in an append-only activity history. A background reminder engine has been provided, which runs at start-up and then every sixty seconds, so that the assignee, the additional recipients and the creator are reminded before the deadline, on the deadline day and at a daily or weekly interval while the task remains overdue, and duplicate reminders have been prevented by a unique database index.

The application has been implemented as a three-tier single page application in which each tier has been built with a modern and freely available technology. The front end has been developed with Angular 19 and TypeScript using standalone components, route guards, HTTP interceptors, Chart.js charts and the Angular CDK drag-and-drop Kanban board. The back end has been developed as an ASP.NET Core Web API on .NET 10 using C#, with Entity Framework Core 10 as the object relational mapper and SQLite as the database, and one hundred and fifteen REST actions have been exposed through twenty-one controllers under the /api/v1 path. Security has been provided through JWT bearer access tokens, rotating refresh-token cookies, PBKDF2 password hashing, mandatory two-step verification with time-based one-time passwords and backup codes, and a permission catalogue of seventy permission keys that have been grouped into system roles and configurable custom roles.

Apart from the core task functions, role-specific dashboards with analytics and filters have been provided, together with employee, department and project management with allocation percentages, an employee workload screen, a notification centre and an internal mail module that has been designed to work without any external mail server. Task reports with

CSV export, reminder and application settings, and a self-service profile with password change and two-step verification management have also been included in the system. An AI based assistant named TFA has been added, which has been designed to answer questions about TaskFlow from the project documents through a hybrid vector and keyword search and to prepare actions such as drafting a mail that are executed only after the user has confirmed them. The system has been verified with more than one thousand unit tests, several hundred integration tests and frontend specification tests, and the results have shown that the stated objectives have been met.

CHAPTER 1
INTRODUCTION

Every organisation, irrespective of its size, has always depended on the timely completion of a large number of small and large tasks by its employees, and the success of the organisation has been closely linked to how well this work is managed. When the allocation of these tasks, the follow-up of their progress and the reminders about their deadlines have been left to individual memory, informal chat messages and loosely maintained spreadsheets, work has frequently been delayed, duplicated or forgotten. The present project, titled TaskFlow – Intelligent Task Allocation and Tracking System, has been undertaken to provide a single, secure and easy to use web application in which tasks are created, assigned, tracked, discussed, reminded and reported in a disciplined manner. The overall system and the profile of the environment in which the project has been carried out have been introduced in this chapter.

1.1 SYSTEM OVERVIEW

TaskFlow has been developed as a web-based office task management application in which tasks are created by managers, assigned to employees with priorities, start dates, deadlines and estimated hours, and followed until the work has been completed. The employees have been given a view of their own tasks, in which the status of each task is updated, the hours that have been worked are logged and the work is discussed with the manager and the other recipients through task comments. Every person who has been involved in a task is reminded automatically before the deadline, on the deadline day and while the task remains overdue, and dashboards, workload figures, activity history and reports have been provided so that pending and late work is identified early. Three system roles, namely Administrator, Manager and Employee, have been provided, and additional custom roles can be created by an administrator from a fixed catalogue of permissions that has been defined in the system.

The application has been built as a three-tier single page application in which each tier has been given a clearly separated responsibility. The presentation tier has been developed with Angular 19 and TypeScript and has been designed to run entirely in the web browser of the user, while the application tier has been developed as an ASP.NET Core Web API on .NET 10 using the C# language. The data tier has been formed by a single SQLite database file that is created and upgraded automatically through Entity Framework Core code-first migrations whenever the application has been started. Two hosted background services have been placed inside the API process, the first of which has been designed to generate deadline reminders every sixty seconds and the second of which has been designed to index the project documents for the built-in assistant. All communication between the browser and the server has been carried out

through JSON over HTTPS, and every request has been authenticated with a JSON Web Token and authorised against the permissions of the signed-in user.

The functional scope of the system has been organised into ten modules, namely Authentication, Dashboard, TaskFlow Monitoring, TaskFlow Management, Communication, Reports, Settings, the personal profile module, the TFA Assistant and the Legal pages. Sign-in with mandatory two-step verification and self-service password recovery through an authenticator application have been provided by the Authentication module. The task list, task creation, task details, reassignment, comments, the activity history, the Kanban board and the employee workload view have been covered by the Monitoring module, whereas employees, departments, projects and positions have been maintained by the Management module. In-app notifications, the automatic reminder engine and an internal mail box have been offered by the Communication module, and reports with CSV export, configurable settings, the personal profile and an AI based assistant that answers questions from the project documents have been provided by the remaining modules.

The objectives that have guided the design of TaskFlow have been stated precisely so that the finished system could be verified against them at the end of the project. The system has been expected to simplify the allocation of tasks to employees, to monitor the progress of every assigned task, to reduce missed deadlines through automatic reminders, and to notify the assignee and the selected recipients about approaching deadlines. It has also been expected to help managers to identify pending, overdue and unevenly distributed work, to improve the communication between managers and employees, to maintain a complete history of task updates, and to generate reports for better work planning. In addition, the protection of organisational data through two-step verification and permission based access, and the configuration of the system without any change to the program code, have been treated as essential objectives of the project.

1.2 ORGANIZATION PROFILE

The project has been carried out as an individual academic mini project in partial fulfilment of the requirements for the award of the degree of Master of Computer Applications of the Centre for Distance and Online Education, Bharathiar University, Coimbatore. Bharathiar University has been established as a State University, has been recognised as a Category-I University by the University Grants Commission, has been accredited with the A++ grade by NAAC, and has been ranked among the leading universities of the country in the NIRF rankings of the Ministry of Education. Professional programmes such as the MCA have been offered by the Centre for Distance and Online Education through the distance and online mode, and its students have been encouraged to apply the theory that has been studied to a realistic software development problem through the mini project.

TaskFlow has been conceived as a personal skill development project that has been aimed at small and medium organisations such as software service companies, consultancy firms, educational institutions and administrative offices, in which work is allocated by a few managers to a team of employees. Such organisations have usually been unable to afford large commercial project management suites and the associated licence fees, and they have often had no dedicated information technology staff to maintain complex installations. The system has therefore been designed to run as a single application server with a file based database, to need no external mail server, and to be configured entirely through its own settings screens. The complete life cycle of the project, from the business requirement document through the functional, high level, low level and technical design documents to the code and the automated tests, has been followed in a structured manner.

CHAPTER 2
SYSTEM STUDY AND ANALYSIS

System study and analysis has been regarded as the phase in which the present method of working is examined in detail, its problems are identified and the requirements of the new system are determined before any design work has been started. During this phase, the way in which tasks were allocated and followed up in a typical office has been studied, the information needs of administrators, managers and employees have been collected, and the business requirements have been recorded in an approved Business Requirements Document together with sixteen sub-module documents. The problem statement, the existing system and its drawbacks, the proposed system and its advantages, and the feasibility of the proposed system from the technical, economic and operational points of view have been described in the sections of this chapter.

2.1 PROBLEM STATEMENT

Without a shared system, tasks have been handed out by word of mouth, through chat applications or through e-mail, and no single list that showed who was doing what and by when has been available to anyone in the organisation. Deadlines have been remembered or forgotten by individuals, and late work has usually been noticed by managers only after the deadline had already passed. Workload has not been visible, and therefore some employees have been overloaded while others have been left waiting for work. Changes that were made to a task, such as a new deadline or a new assignee, have not been recorded anywhere, and reports for planning have had to be prepared by counting tasks manually. Access to information has also not been controlled properly, since everything could be seen by anyone who knew a shared password, and a stolen password has been sufficient for an outsider to gain entry.

The problem that has been addressed by this project has therefore been stated as the need for a secure, role-based and centrally maintained web application in which every task is recorded with its assignee, priority, start date, deadline and estimated effort, in which a well-defined status life cycle is enforced, and in which all the concerned people are reminded automatically and exactly once at each reminder point. The application has been required to keep a complete and unalterable history of every change, to provide each user with a dashboard that has been limited to the data which the user is permitted to see, to show the workload of every employee, and to produce filtered reports that can be exported for further analysis. It has also been required to protect every account with two-step verification and to work without depending on an external e-mail server.

2.2 EXISTING SYSTEM

In the existing manual system that has been studied, the work that had to be done was decided by a manager and communicated to an employee verbally, through a chat message or through an e-mail, and in some cases the task was also noted in a shared spreadsheet that was maintained by the manager. A personal note of the task and its deadline was kept by the employee, and progress was reported only when it had been asked for by the manager in a meeting or through another message. Reminders were given manually by the manager whenever the deadline was remembered, and any change in the deadline or the assignee was communicated through further messages that were scattered across different applications. At the end of the month, a report was prepared by the manager by going through the spreadsheet and the messages and by counting the completed, pending and delayed tasks by hand.

2.2.1. Drawbacks

The study of the existing system has revealed a number of drawbacks that have directly affected the productivity of the team and the quality of the decisions that have been taken by the management. The information was gathered by observing the daily work of a small team, by examining the spreadsheets and messages that were used for allocating work, and by discussing the difficulties with managers and employees, so that the drawbacks have been identified from actual practice rather than from assumptions. The most important drawbacks that have been identified are listed below, and each of them has been treated as a requirement that had to be overcome by the proposed system before it could be accepted by the users.

- No single and reliable list of tasks has been available, and therefore the status of a task has had to be found out by asking the concerned employee, which has wasted the time of both the manager and the employee.
- Reminders have depended entirely on the memory of individuals, and as a result deadlines have frequently been missed and the delay has been noticed by the manager only after the work had already become late.
- The workload of each employee has not been visible, so work has been distributed unevenly and some employees have been overloaded while others have remained idle for long periods.
- Changes such as a revised deadline, a change of assignee or a change of priority have not been recorded, and the reason for a reassignment has been lost, which has made later review and accountability impossible.
- Reports have had to be prepared manually by counting tasks in spreadsheets and messages, which has been slow, error prone and not repeatable for different periods, projects or employees.

- Access to the shared spreadsheet has not been controlled according to the role of the user, and access to all the information has been granted by a single password without any second factor of verification.

2.3 PROPOSED SYSTEM

In the proposed system, TaskFlow, the informal manual process has been replaced with a centralised web application in which every task is created on one screen with its title, description, project, assignee, priority, start date, deadline, estimated hours, additional recipients and reminder intervals, and is saved with a unique task code such as TSK-000042. Each task has been made to move through a fixed set of six statuses, namely To Do, In Progress, In QA, Blocked, Completed and Cancelled, and only the permitted transitions have been accepted by the server. Overdue has not been stored as a status but has been derived at the time of reading from the deadline and the status, so that it can never become stale. Every important change has been recorded with the user who has made it, the time and the old and new values, and comments and worked hours have been maintained against each task.

A background reminder engine has been provided, in which all the open tasks are examined every sixty seconds and it is decided, in the configured application time zone, whether a first reminder, a second reminder, a due-today reminder or an overdue reminder is due on the current day. The reminder has been delivered as an in-app notification and, when the corresponding setting has been enabled, also as a system mail in the internal mail box of every recipient, and a unique database index has guaranteed that each reminder is delivered exactly once even when the server has been restarted. Dashboards and analytics have been provided at three scopes, namely Own, Team and Organisation, and every screen, menu item and button has been shown according to the permissions of the user. Sign-in has been protected by a password and a time-based one-time code from an authenticator application, and an AI based assistant has been added to help users to find information and to perform simple actions after confirmation.

2.3.1. Advantages

The proposed system has been designed so that each drawback of the existing system is removed and additional benefits are obtained for the organisation, the administrators, the managers and the employees who have been identified as its users. Several of these advantages have been obtained without any additional effort from the users, because reminders, history entries and notifications have been generated automatically by the system as a side effect of the normal work. In addition, the system has been built entirely with free and open source technologies, so that these advantages have been obtained without any licence cost to the organisation. The principal advantages that have been realised in the developed system have been summarised in the list that is given below.

- A task can be created and assigned by a manager on a single screen, and the assignee has been notified as soon as the task has been saved, which has simplified and speeded up the allocation of work.
- Automatic reminders have been delivered before the deadline, on the deadline day and while the task is overdue, so that missed deadlines have been reduced without any manual follow-up by the manager.
- Role-specific dashboards, the Kanban board and the employee workload screen have made pending, overdue and unevenly distributed work visible immediately to the people who have been made responsible for it.
- Every change to a task has been recorded in an append-only activity history together with the reason for every reassignment, which has provided complete traceability and accountability.
- Reports have been generated instantly for any combination of period, project, employee, status and priority, and the filtered tasks can be exported to a CSV file that has been designed for further analysis.
- Data has been protected through mandatory two-step verification, permission based access with Own, Team and Organisation scopes, encrypted secrets and sessions that have been ended automatically after any security change.
- Reminder defaults, the time zone, the default priority, notifications, roles and the AI provider have been made changeable from the settings screens without any modification of the program code.

2.4 FEASIBILITY ANALYSIS

A feasibility analysis has been conducted in order to determine whether the proposed system could be developed and used successfully with the available resources, within the available time and at an acceptable cost. The proposal has been examined from the technical, economic and operational points of view, and the cost and the schedule of the project have also been estimated as a part of the same study. The conclusion of each part of the study has been found to be favourable, and on that basis the development of TaskFlow has been taken up without any change to the scope that had been approved. The individual aspects of the study have been described in the following sub-sections, together with the cost estimate and the schedule that have been followed for the project.

2.4.1. Technical Feasibility

The technical feasibility study has examined whether the technology that was required for the proposed system was available, mature and within the skills of the developer. The system has been planned with ASP.NET Core on .NET 10, Entity Framework Core, SQLite and

Angular 19, all of which have been recognised as stable, well documented and freely available technologies that are supported by large developer communities and that run on Windows, Linux and macOS. The hosted BackgroundService feature of ASP.NET Core has been found to be adequate for the reminder scheduler without any external message broker, and SQLite has been found to be adequate for a single server deployment with a modest number of concurrent users. Since a standard laptop has been sufficient for development and testing and the required libraries could be obtained through NuGet and npm, the proposed system has been found to be technically feasible.

2.4.2. Economic Feasibility

The economic feasibility study has compared the cost of developing and operating the proposed system with the benefits that have been expected from it. All the development tools, frameworks and libraries that have been used, including the .NET SDK, Visual Studio Code, Angular, Node.js, SQLite and the testing frameworks, have been obtained as open source or free of charge, and therefore no licence cost has been incurred. The system has been designed to run on a single server with a file based database, so that no separate database server, mail server or paid cloud service has been required, and the only direct software expenditure has been the usage charge of about Rs. 3,000 that has been paid for the AI provider which has been used by the TFA assistant during development and testing. Since the benefits in the form of fewer missed deadlines, less time spent on manual follow-up and instant reports have far exceeded this small expenditure and the maintenance cost, the system has been found to be economically feasible.

2.4.3. Operational Feasibility

The operational feasibility study has examined whether the proposed system would be accepted and used effectively by the people for whom it has been built. The users of the system have been identified as administrators, managers and employees who are already familiar with web browsers, and the screens of TaskFlow have been designed with familiar elements such as lists, forms, cards, charts and a drag-and-drop board, together with clear validation messages and confirmation dialogs. Only the menu items and data that are relevant to the role have been shown to each user, which has kept the interface simple, and the reminders, notifications and internal mail have been made to work automatically without any training. Because the routine effort of the managers has been reduced and the employees have been given a clear personal list of work, resistance to its introduction has been expected to be low, and the system has been found to be operationally feasible.

2.4.4. Cost Estimation And Scheduling

The cost of the project has been estimated by considering the hardware, the software, the AI provider, the development effort and the operating expenses, and the resulting estimate

has been shown in Table 2.1. Since the existing laptop of the developer has been used and all the software has been obtained free of charge, the direct expenditure has been limited to the AI provider charge of Rs. 3,000 and to the internet and electricity charges, while the development effort has been valued notionally in order to indicate the economic worth of the system. The effort has been calculated from the actual schedule of about thirty working days, and a notional daily rate has been applied only for the purpose of comparison with commercial development, while no separate expenditure has been incurred for printing or binding because the report has been prepared and submitted as an electronic document.

Table 2.1 Estimated Cost Of The Project

S.No.	Item	Basis Of Estimate	Cost (Rs.)
1	Development hardware (existing laptop)	Already available, no purchase	0
2	Software tools and frameworks	Open source and free licences	0
3	AI provider for the TFA assistant	AI API usage charges	3,000
4	Internet and electricity during development	30 days at about Rs. 50 per day	1,500
5	Development effort (notional)	30 person-days at Rs. 1,000 per day	30,000
	Total estimated cost		**34,500**

The scheduling of the project has been carried out by following a documentation-first sequence in which every phase has been completed before the next one was started, and the resulting schedule has been shown as a Gantt chart in Figure 2.1. The Business Requirements Document has been prepared first, followed by the Functional Requirements Document, the Technical Design Document, the High-Level Design and the Low-Level Design, after which the OpenSpec specifications and the change artifacts, namely the proposal, the design and the task list of each change, have been written. Only after these artifacts had been approved has the coding been started on 11 October 2026, and the modules have been developed one after another while unit and integration testing has been carried out continuously alongside them, so that defects have been detected soon after they had been introduced. The project report has been prepared during the final days, and the whole project has been completed in about six weeks, which has been within the time that had been allotted for the mini project.

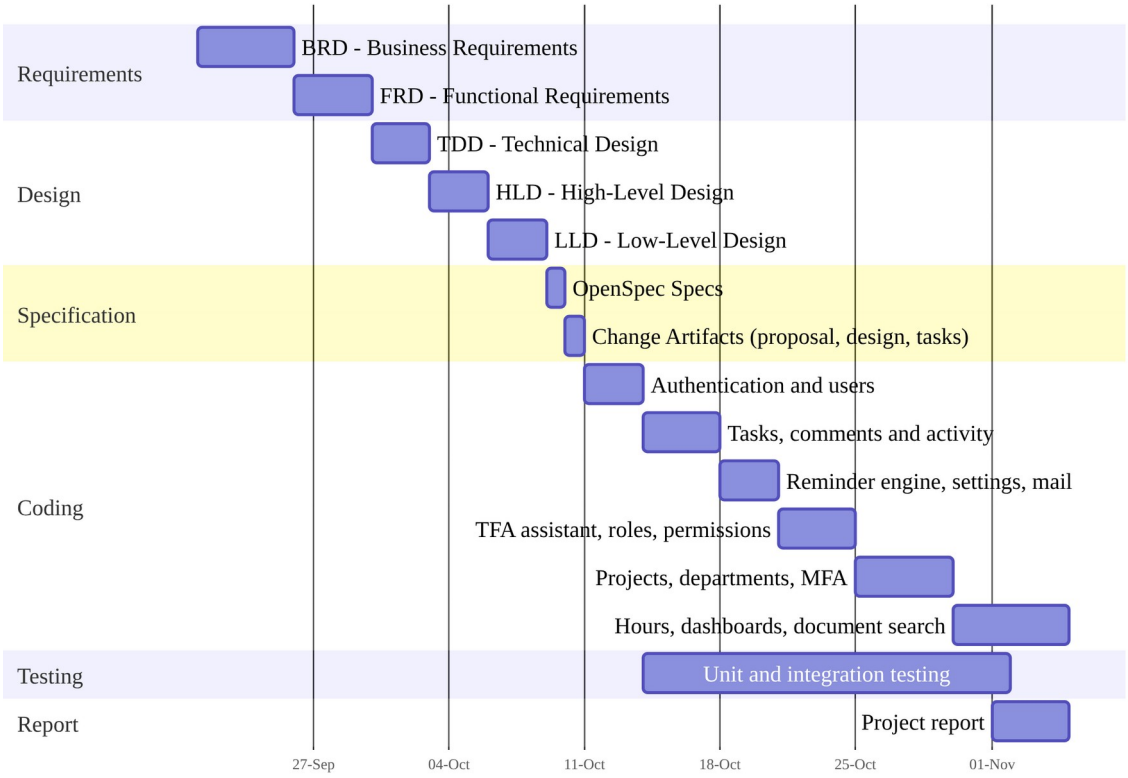


Figure 2.1 Gantt Chart Of The Project Schedule

CHAPTER 3
DEVELOPMENT ENVIRONMENT

The development environment of a project has been understood as the combination of the hardware, the operating system, the programming languages, the frameworks, the database and the tools that have been used to design, build, test and run the system. The choice of this environment has been made carefully, because it has a direct effect on the productivity of the developer, on the performance and the security of the finished application and on the ease with which the application can be maintained in future. For TaskFlow, a modern and fully open source environment has been chosen, in which C# on ASP.NET Core and .NET 10 has been used for the server, TypeScript on Angular 19 has been used for the browser, and SQLite with Entity Framework Core has been used for the data, and the requirements and the main technologies that have been used are described in the sections of this chapter.

3.1 HARDWARE REQUIREMENT

The hardware requirement has been specified separately for the development machine, on which the source code has been written, compiled and tested, and for the server and client machines, on which the finished system will be run by the organisation. Since the API and the reminder scheduler have been designed to run in a single process with a file based SQLite database, the server requirement has been kept modest, and an ordinary office computer or a small virtual machine has been found to be sufficient for a team of a few hundred users. The client has been required to have only a modern web browser, because all the processing of the user interface has been carried out inside the browser itself, and the minimum hardware configuration that has been recommended for both machines is listed in Table 3.1.

Table 3.1 Hardware Requirement

Component	Development / Server Machine	Client Machine
Processor	Intel Core i5 or AMD Ryzen 5, 64-bit (x64)	Any dual core processor
Main memory (RAM)	8 GB minimum, 16 GB recommended	4 GB
Hard disk	10 GB free space (SSD preferred)	Not applicable
Display	1366 x 768 or higher resolution	1366 x 768 or higher resolution
Network	Broadband internet for packages and AI provider	LAN or internet connection
Input devices	Standard keyboard and mouse	Keyboard, mouse; microphone for voice input
Mobile device	Smartphone with an authenticator application	Smartphone with an authenticator application

3.2 SOFTWARE REQUIREMENT

The software requirement has been prepared as a list of the operating system, the runtime, the languages, the frameworks, the database and the tools that have been needed to build and run TaskFlow on a new machine. All the items in the list have been chosen from open source or free of charge products, and the exact versions that have been used during development have been recorded so that the build can be reproduced later on another machine without any surprise. The back end has been targeted at the net10.0 framework, the front end has been built with Angular 19.2 and TypeScript 5.7, and the database has been accessed through Entity Framework Core 10.0.12 with the SQLite provider, and the complete software requirement of the system has been summarised in Table 3.2.

Table 3.2 Software Requirement

Category	Software And Version
Operating system	Windows 10 / 11 (64-bit); Linux and macOS are also supported
Server runtime	.NET 10 SDK and ASP.NET Core 10 runtime
Server side language	C# 14 with nullable reference types
Web framework	ASP.NET Core Web API with JwtBearer 10.0.12, FluentValidation 12.2.0, SpaProxy 10.0.12
ORM and database	Entity Framework Core 10.0.12 with Microsoft.EntityFrameworkCore.Sqlite; SQLite 3
Client side framework	Angular 19.2 (standalone components), RxJS 7.8, Angular CDK 19
Client side language	TypeScript 5.7, HTML5, CSS3
Charts and utilities	Chart.js 4.6 with ng2-charts 8, qrcode 1.5
Package managers	NuGet, Node.js 22 LTS with npm
IDE and tools	Visual Studio 2022 / 2026, Visual Studio Code, Git, Angular CLI
Testing tools	xUnit, WebApplicationFactory, NetArchTest, Jasmine and Karma
AI provider (optional)	Google Gemini (default), OpenAI or local Ollama
Client browser	Google Chrome, Microsoft Edge, Mozilla Firefox (latest versions)

3.3 PROGRAMMING ENVIRONMENT

The programming environment of TaskFlow has been organised as one repository in which the back end solution, the front end workspace, the design documents and the change proposals have all been kept together under version control. The back end solution, TaskFlow.sln, has been made up of one API project named TaskFlow.Api together with a unit test project and an integration test project, while the Angular workspace has been kept in a separate frontend folder of the same repository. The API project has been given a reference to Microsoft.AspNetCore.SpaProxy, so that when F5 has been pressed in Visual Studio or the dotnet run command has been executed, the API has been started on https://localhost:7055 and the Angular development server has been launched automatically on http://localhost:4200, and the technologies that have formed this programming environment are described in the sub-sections that follow.

3.3.1. About ASP.NET Core And C#

C# has been developed by Microsoft as a modern, object oriented and type safe programming language, and it has been standardised by ECMA and ISO with support for classes, interfaces, generics, records, pattern matching, asynchronous programming with async and await, and language integrated queries through LINQ. ASP.NET Core has been released

by Microsoft as the open source and cross-platform web framework of .NET, in which web APIs, web applications and real-time services can be built, and in which a configurable middleware pipeline, built-in dependency injection, configuration from several sources, logging, authentication, authorisation and hosted background services have been provided. In TaskFlow, ASP.NET Core has been used to implement twenty-one controllers that have exposed one hundred and fifteen REST actions, and the business rules have been kept in service classes which have been injected into the controllers through interfaces.

Several features of ASP.NET Core have been used extensively in the project, and each of them has been chosen because it has solved a specific requirement without any additional third party library. The JwtBearer authentication handler has been used to validate the access token of every request, the policy based authorisation system has been used to check one policy for each of the seventy permission keys, and a resource based handler has been written to check whether the caller has been allowed to access a particular task. The ProblemDetails support of the framework has been combined with a global exception handler so that every error has been returned in the standard RFC 7807 format, and the BackgroundService base class with a PeriodicTimer has been used to implement the reminder scheduler. ASP.NET Core Data Protection, which has used DPAPI on Windows, has been applied to encrypt the authenticator secrets of the users and the API keys of the AI providers, and the built-in PasswordHasher has been used to store every password as a salted PBKDF2 hash.

3.3.2. About Angular And TypeScript

Angular has been developed and maintained by Google as an open source, component based framework for building single page web applications, and TypeScript has been developed by Microsoft as a statically typed superset of JavaScript which has been compiled into plain JavaScript that runs in every browser. In Angular, a component model with HTML templates and data binding, a powerful router with lazy loading and route guards, an HTTP client with interceptors, reactive forms with validation and a dependency injection system have been provided, while compile time type checking, interfaces and modern language features have been added by TypeScript so that large front end code bases have become easier to maintain. In TaskFlow, Angular 19 has been used with standalone components, and the application has been divided into lazy loaded feature modules for authentication, dashboard, management, monitoring, communication, reports, profile, settings, assistant and legal pages.

The authentication interceptor of the front end has been written to attach the bearer token to every API call, to refresh the token silently once when the server has returned the 401 status, and to redirect the user to the sign-in page when the refresh has failed. Route guards such as authGuard, mfaSetupGuard and permissionGuard have been used to prevent a user from opening a screen for which the required permission has not been granted, although the same checks have always been repeated by the server. Angular signals have been used for the

local state of the screens and RxJS observables have been used for the HTTP calls, the charts of the dashboards have been drawn with Chart.js through the ng2-charts wrapper, and the drag-and-drop Kanban board has been implemented with the Angular Component Dev Kit. The styling has been done with custom CSS design tokens and primitives, and no external CSS framework has been used, which has kept the size of the bundle small for the users of the application.

3.3.3. About SQLite And Entity Framework Core

SQLite has been designed as a self-contained, serverless, zero configuration and transactional SQL database engine in which the complete database, including all the tables, indexes and triggers, has been stored in one ordinary disk file. Most of the SQL standard has been implemented in SQLite together with ACID transactions, unique and check constraints, foreign keys, triggers and the FTS5 full text search extension, and it has become one of the most widely deployed database engines in the world. Entity Framework Core has been provided by Microsoft as the object relational mapper of .NET, in which the database has been described through C# entity classes and a DbContext, LINQ queries have been translated into parameterised SQL statements, the changes made to the entities have been tracked and saved in a single transaction, and migrations that create and upgrade the database schema have been generated automatically.

In TaskFlow, the database has been designed with the code-first approach, in which thirty-three entity classes and their relationships have been defined in the AppDbContext class and twenty-four migrations have been generated as the design has evolved over the weeks of development. The migrations have been applied automatically at start-up through the MigrateAsync method, after which seeders have inserted the permission catalogue, the system roles, the default settings and the built-in AI models into the database. The database file, taskflow.db, has been kept together with the encryption key ring and the mail attachments in a single data folder, which has been set to C:\TaskFlow on Windows, so that the complete data of the organisation can be backed up by copying one folder. SQLite triggers have been used to make the system roles and the permission catalogue read-only, and a unique index has been used to prevent duplicate reminders from being delivered to any recipient.

CHAPTER 4
SYSTEM DESIGN AND DEVELOPMENT

System design has been regarded as the phase in which the requirements that had been identified during analysis were translated into a blueprint of the processes, the data, the interfaces and the architecture of the new system, and system development has been regarded as the phase in which that blueprint was turned into working and tested program code. For TaskFlow, the design has been recorded in a set of documents which have been prepared in the order of the Business Requirements Document, the Functional Requirements Document, the Technical Design Document, the High-Level Design and the Low-Level Design, followed by the OpenSpec specifications and the change artifacts, and the code has been written only after these documents had been completed so that they were followed closely. The elements of the design, namely the process, concept, logical, physical, input, output and database design, have been presented in this chapter, and they have been followed by the structure of the main database tables of the system.

4.1 ELEMENTS OF DESIGN

The design of TaskFlow has been approached from several complementary views so that every aspect of the system could be specified before it was coded by the developer. The process design has described how data was moved through the system and how each important operation was carried out, the concept design has described the main ideas and rules of the problem domain, and the logical design has described the layers, components and interfaces of the software. The physical design has described how the system was deployed and where its data was stored, the input and output designs have described the screens through which users interacted with the system, and the database design has described the entities, relationships and constraints of the data. Each of these elements has been presented in the sub-sections below with the help of diagrams and tables that have been prepared from the actual code.

4.1.1. Process Design

The process design of TaskFlow has been started with the overall architecture of the system, which has been shown in Figure 4.1. A three-tier architecture has been followed, in which the Angular single page application has formed the presentation tier, the ASP.NET Core Web API has formed the application tier, and the SQLite database together with the attachment folder and the encryption key ring has formed the data tier. Inside the API, every request has been passed through a middleware pipeline in which HTTPS redirection, CORS checking, JWT authentication, the mandatory two-step verification gate and permission based authorisation were performed, after which the request was handed over to a thin controller. The work has been delegated by the controller to a service in which the business rules were applied, and the

service has used repositories through which the database was queried and changed with Entity Framework Core, while two background services have been run inside the same process.

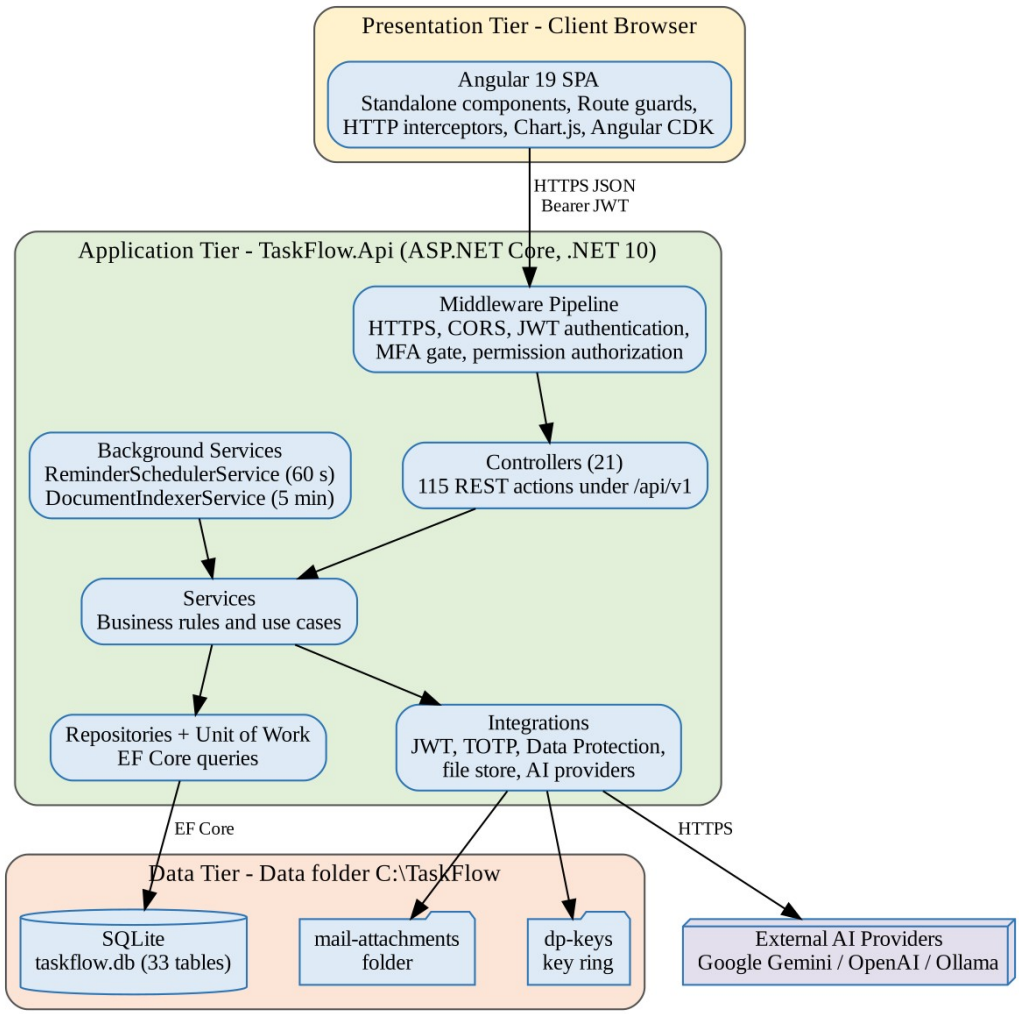


Figure 4.1 System Architecture Of TaskFlow

The functional decomposition of the system into modules and screens has been shown in Figure 4.2, and it has been prepared directly from the side menu and the routes of the application. The application has been divided into nine functional modules that were shown in the side menu, together with the public legal pages, and each module has been further divided into sub-modules that were mapped to one or more screens of the application. The same module tree has been used consistently in every layer of the code, so that the controllers, services, repositories and data transfer objects of a module have been placed in folders with the same name, and the same names have also been used for the permission groups and the design documents. Because of this consistency, each requirement has been traced easily from the business document to the screen, the API endpoint, the service, the table and the automated test by which it was verified.

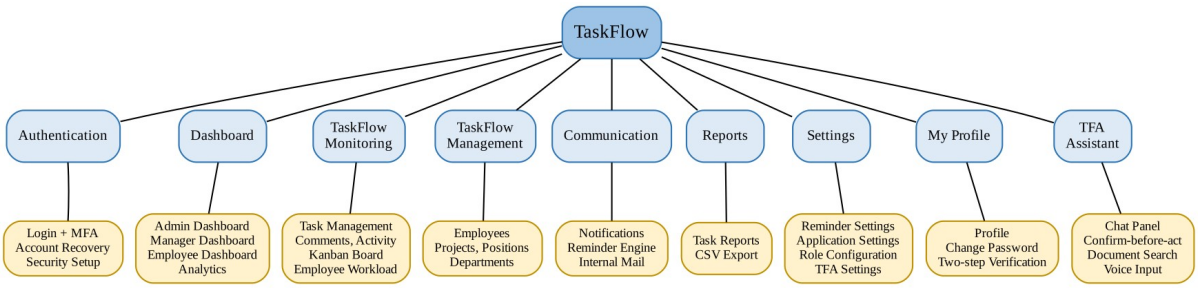


Figure 4.2 Module Hierarchy Of TaskFlow

The context level data flow diagram, which has also been called the level 0 diagram, has been shown in Figure 4.3 for the complete system. In this diagram the complete system has been represented as a single process, and the Administrator, the Manager, the Employee and the external AI provider have been identified as the external entities with which it interacted. Master data such as employees, departments, projects, roles and settings has been supplied by the administrator, who has received the organisation dashboard and reports in return, while tasks, assignments and reassignments have been supplied by the manager, who has received the team dashboard, workload figures and reminders. Status updates, worked hours and comments have been supplied by the employee, to whom the assigned tasks, reminders, notifications and mail were returned, whereas the prompts of the assistant together with the retrieved document context were sent to the AI provider, from which answers or requests for tools were received.

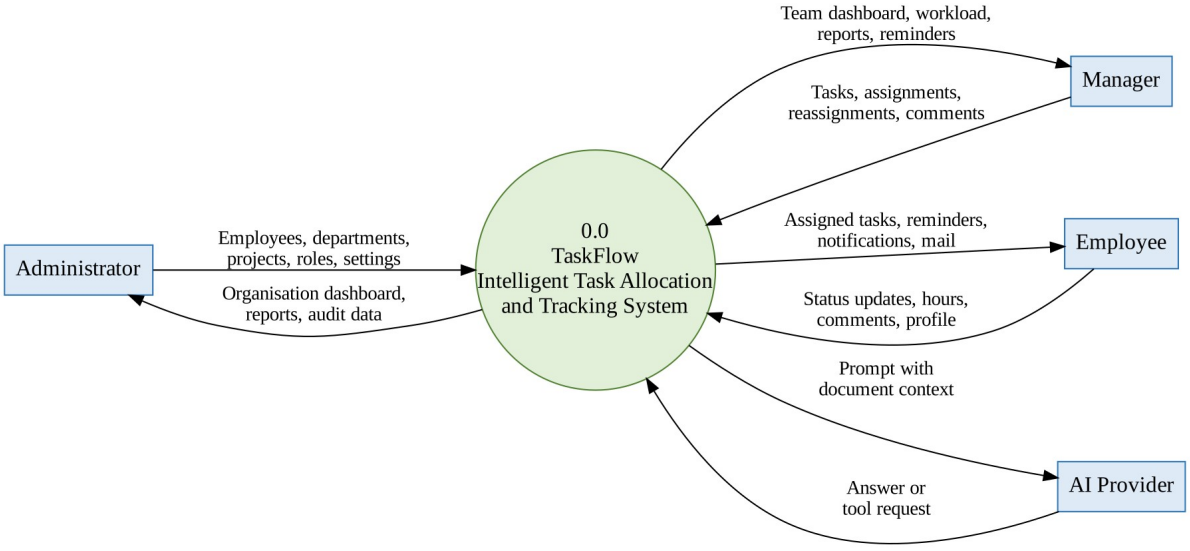


Figure 4.3 Level 0 Data Flow Diagram

The level 1 data flow diagram, which has been shown in Figure 4.4, has expanded the single process of the context diagram into six major processes, namely Authenticate User, Manage Master Data, Manage Tasks, Generate Reminders, Communicate and Generate Reports, and the six main data stores that were used by these processes have also been shown in it. In the authentication process the credentials and the one-time code were verified against the users and roles store and a token was returned, while the users, projects and departments were

maintained by the master data process. Tasks, recipients, comments and activity entries have been stored by the task process, from which event notifications were raised, and the reminder process has been triggered by the scheduler, so that the open tasks and the reminder settings were read and the reminder history and notifications were written. The task data has been read by the reports process according to the filters given by the user, and the summary and the CSV file were returned to the user.

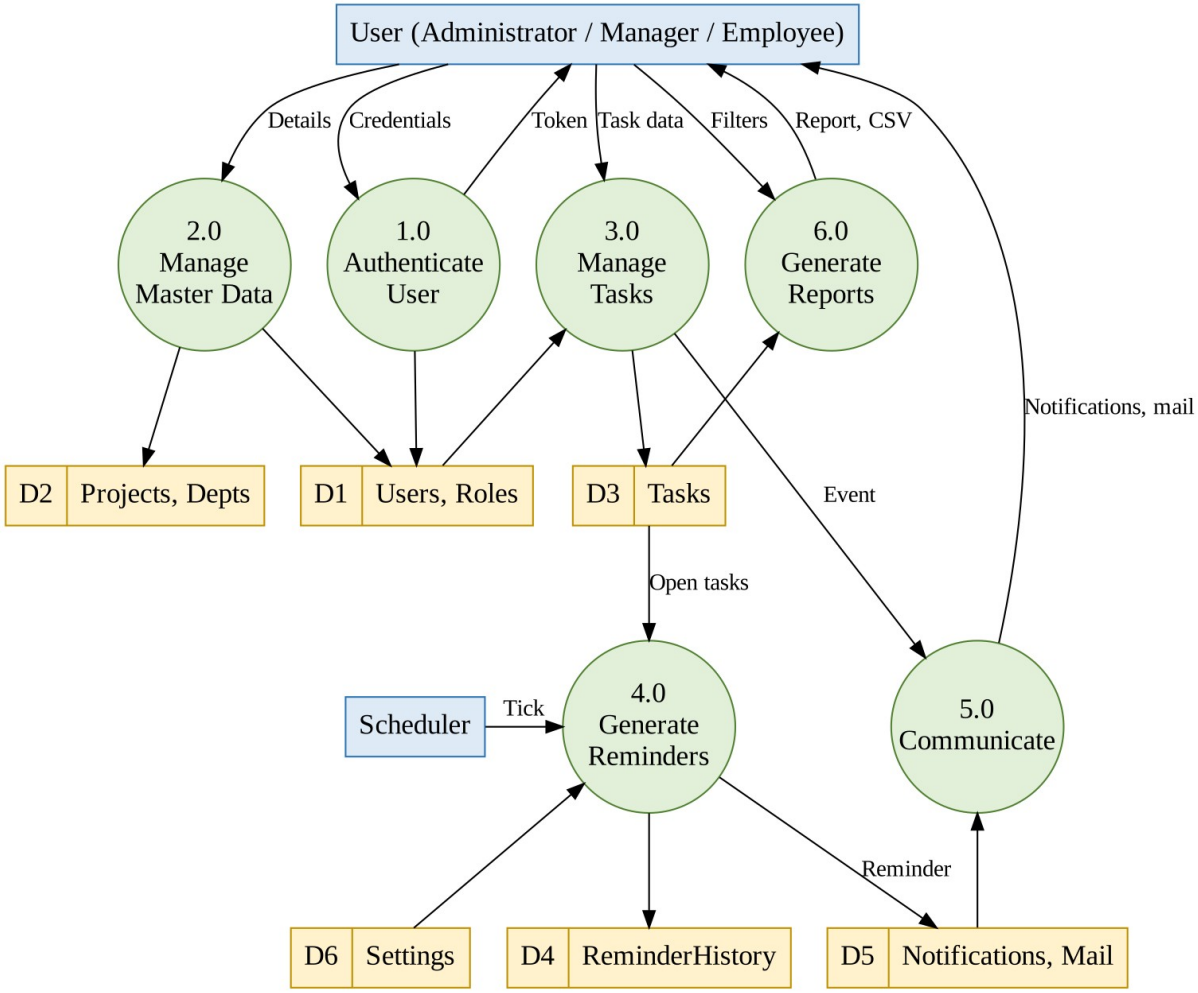


Figure 4.4 Level 1 Data Flow Diagram

The level 2 data flow diagram for the Manage Tasks process has been shown in Figure 4.5, and the five sub-processes through which a task was created, its status was updated, it was reassigned, it was discussed and its worked hours were logged have been described in it. The tasks store and the related recipients, comments and activity log stores have been written by each sub-process, and the notifications store has also been written by the sub-processes whose results were of interest to other people. It has been shown clearly in the diagram that an entry was produced in the activity log for every change to a task, which has been the reason why the complete history of each task could be reconstructed later. Both a new assignee and a reason

have been required by the reassignment sub-process, and the previous assignee has been kept as a recipient so that the previous assignee was still informed about the task.

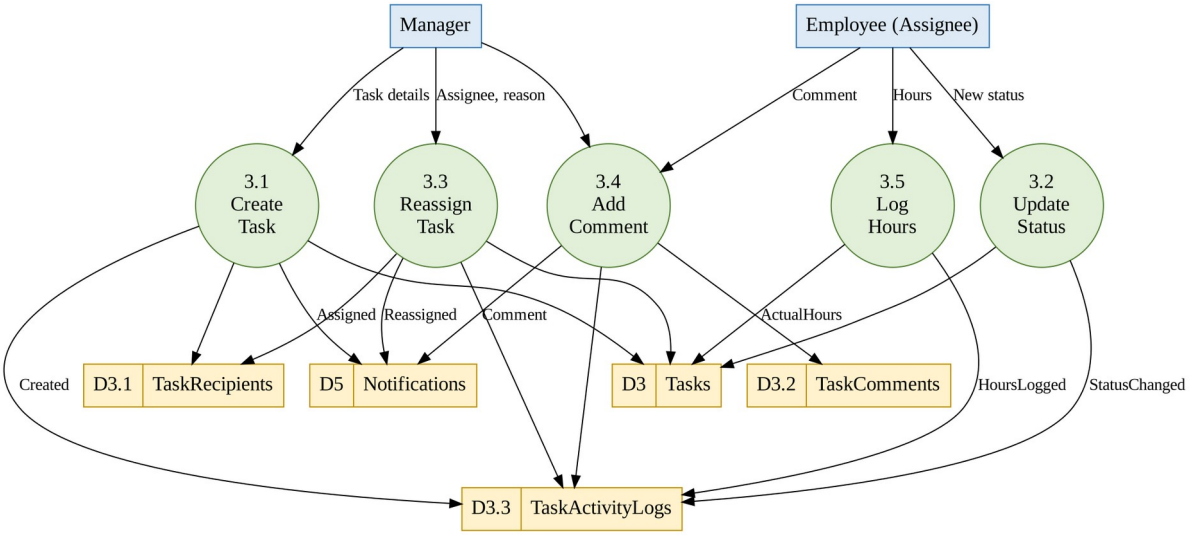


Figure 4.5 Level 2 Data Flow Diagram For Task Management

The use case diagram of the system has been shown in Figure 4.6, in which the three main actors and the principal use cases that were performed by each of them with the default permissions of the system roles have been identified. Employees, departments, projects, roles, permissions and application settings have been managed by the administrator, by whom the organisation dashboard and reports were also viewed. Tasks have been created, assigned and reassigned by the manager, by whom the team workload and reports were viewed, the reminder defaults were set and task discussions were joined, while the status of the assigned tasks has been updated by the employee, who has also logged worked hours, commented on tasks, used the Kanban board, read notifications and mail, managed the profile and asked questions to the TFA assistant. Signing in with two-step verification and viewing a dashboard have been treated as use cases that were common to all the three actors of the system.

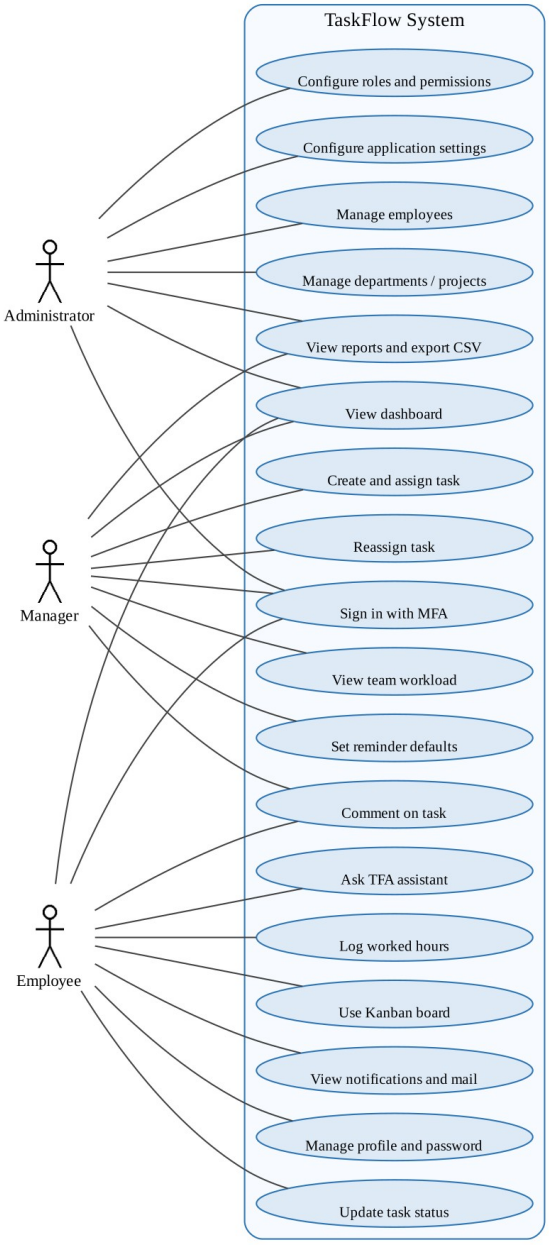


Figure 4.6 Use Case Diagram Of TaskFlow

The flow of every request inside the API has been shown as a sequence diagram in Figure 4.7, in which the interaction between the browser, the pipeline, the controller, the service, the repository and the database has been traced. An HTTPS request that carried the bearer token has been sent by the browser, after which the token was validated, the current role and permissions of the user were reloaded from the database through the session validation step, the two-step verification gate was checked and the permission policy of the endpoint was evaluated. The authorised request has then been passed to the controller, where the FluentValidation validator of the request object was run by a global validation filter, and the service was called with a request data transfer object. The data has been queried or changed by the service through the repository, all the changes have been committed together through the

unit of work, and a response object was returned, which was sent back by the controller as JSON or as a ProblemDetails object when an error had occurred.

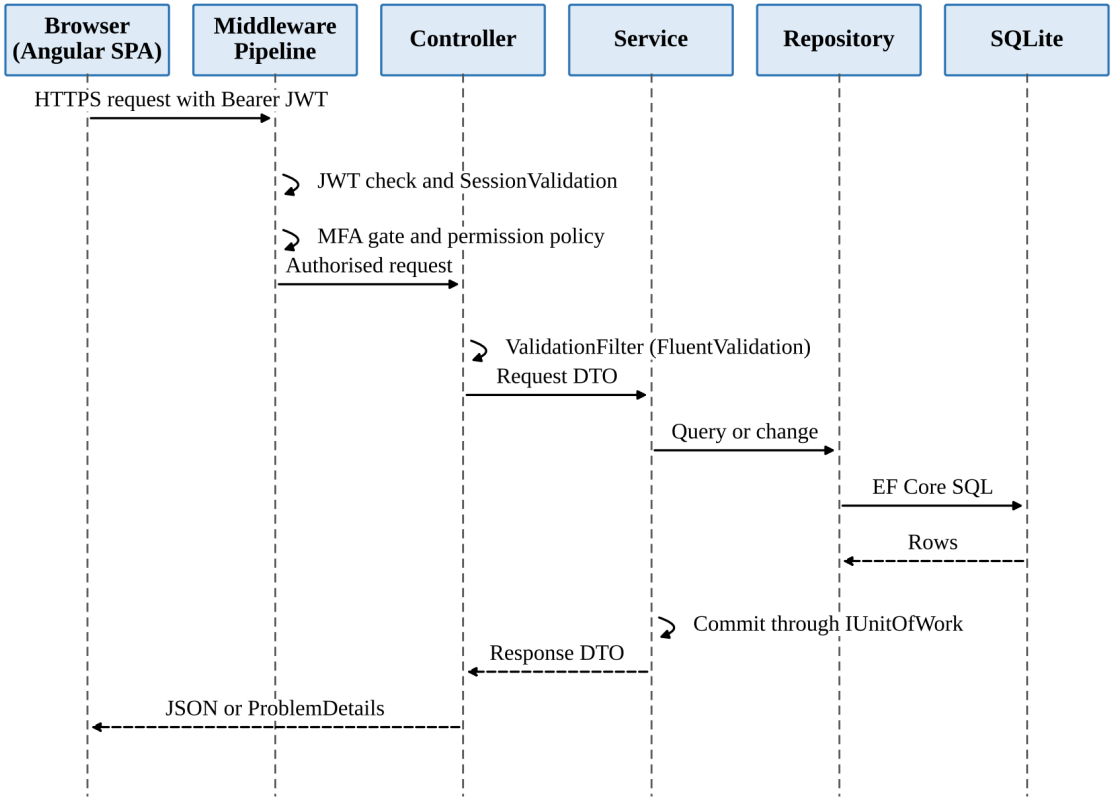


Figure 4.7 Sequence Diagram Of An API Request

The sign-in process with two-step verification has been shown as a flowchart in Figure 4.8, and the flowchart has been prepared from the actual authentication code of the application so that every decision of the server has been represented. An email address or user name together with the password has been entered by the user, and when the account was missing, inactive or the password was wrong, a bare 401 response was returned by the server without revealing which of the three conditions had failed. When the credentials were correct and two-step verification had been enabled, a challenge token that was valid for five minutes has been returned, after which the six-digit code from the authenticator application or one of the ten backup codes was entered by the user. Code entry has been locked for five minutes after five consecutive wrong codes, and when the code was correct the access token and the refresh cookie were issued and the user was taken to the dashboard that matched the role.

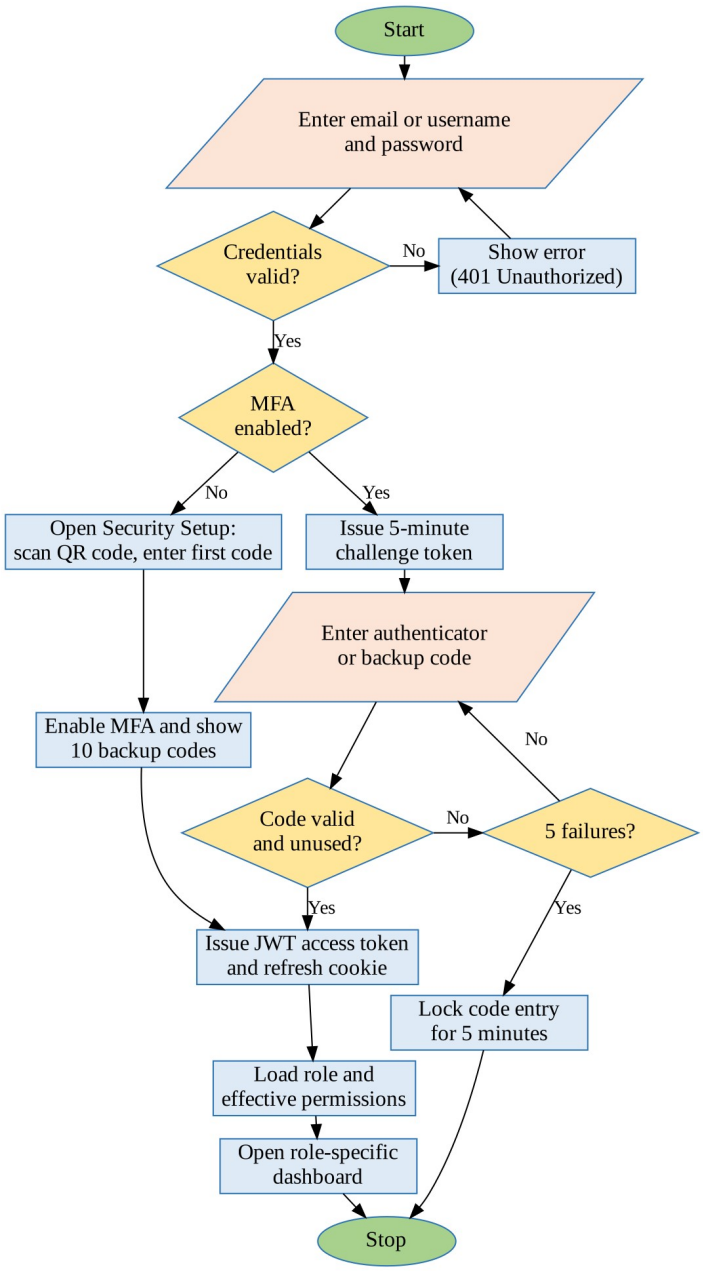


Figure 4.8 Flowchart For Sign-In With Two-Step Verification

The same sign-in process has been drawn as a sequence diagram in Figure 4.9, in which the messages that have been exchanged between the Angular login component, the authentication controller, the authentication service, the MFA service and the SQLite database have been shown in the order in which they were sent. The first request has been sent to the /api/v1/auth/login endpoint, and when two-step verification had been enabled, a response containing the mfaRequired flag and the challenge token has been returned instead of a session. The second request has been sent to the /api/v1/auth/mfa/verify endpoint with the challenge token and the code, after which the code has been verified by the MFA service, the last login time has been set, a refresh token row has been inserted and the authentication response has been returned together with the taskflow_refresh cookie.

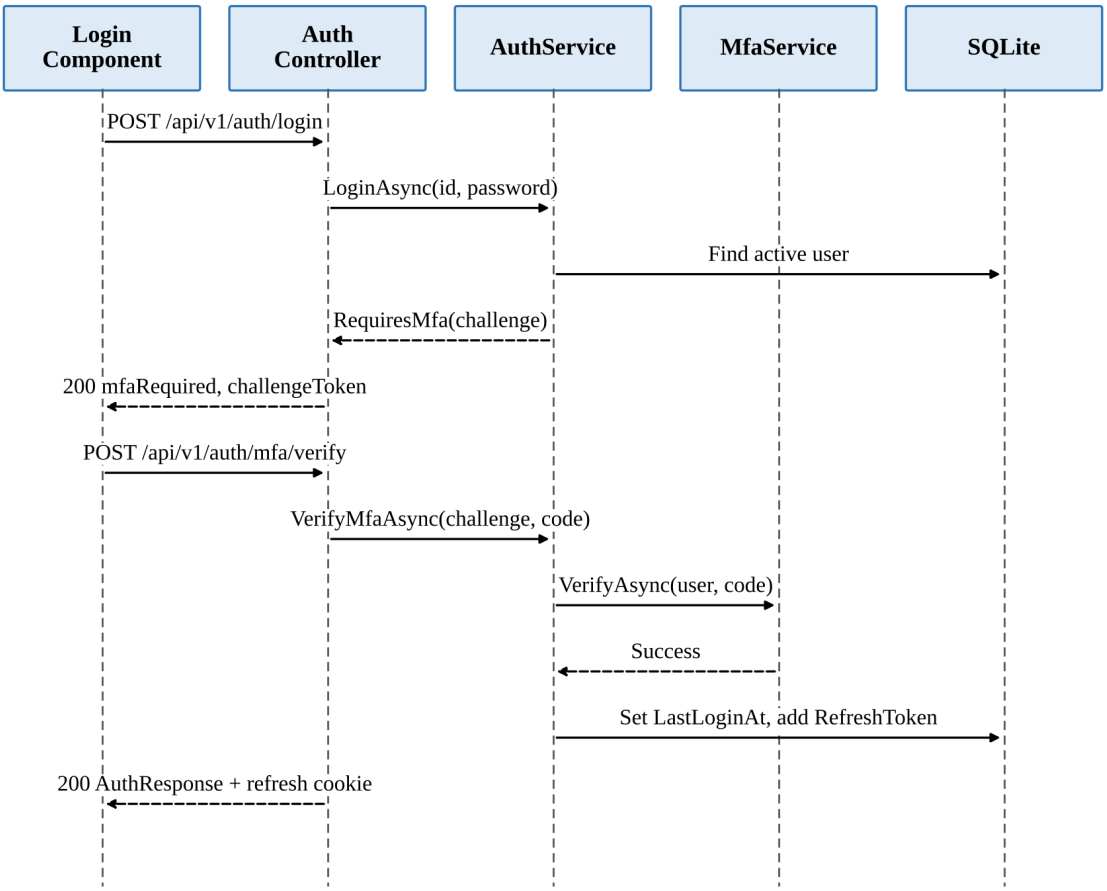


Figure 4.9 Sequence Diagram For Sign-In

The process of creating a task has been shown as a flowchart in Figure 4.10, in which every validation and every database step that was carried out by the server has been represented. The details of the task have been entered by the manager, and the request has first been validated by the server, so that it was checked that the title was present and not longer than two hundred characters, that the deadline was after the start date, that the estimated hours were positive, that there were at most fifty additional recipients and that the two reminder intervals were in the correct order. It has then been verified that the assignee was an active employee and that the project existed, and inside a single database transaction the task was inserted in the To Do status, the task code was generated, the assignee and the additional recipients were added, the Created and Assigned activity entries were written and the Task Assigned notification was published. If any step had failed, the whole transaction would have been rolled back and the reason would have been shown to the user on the same screen.

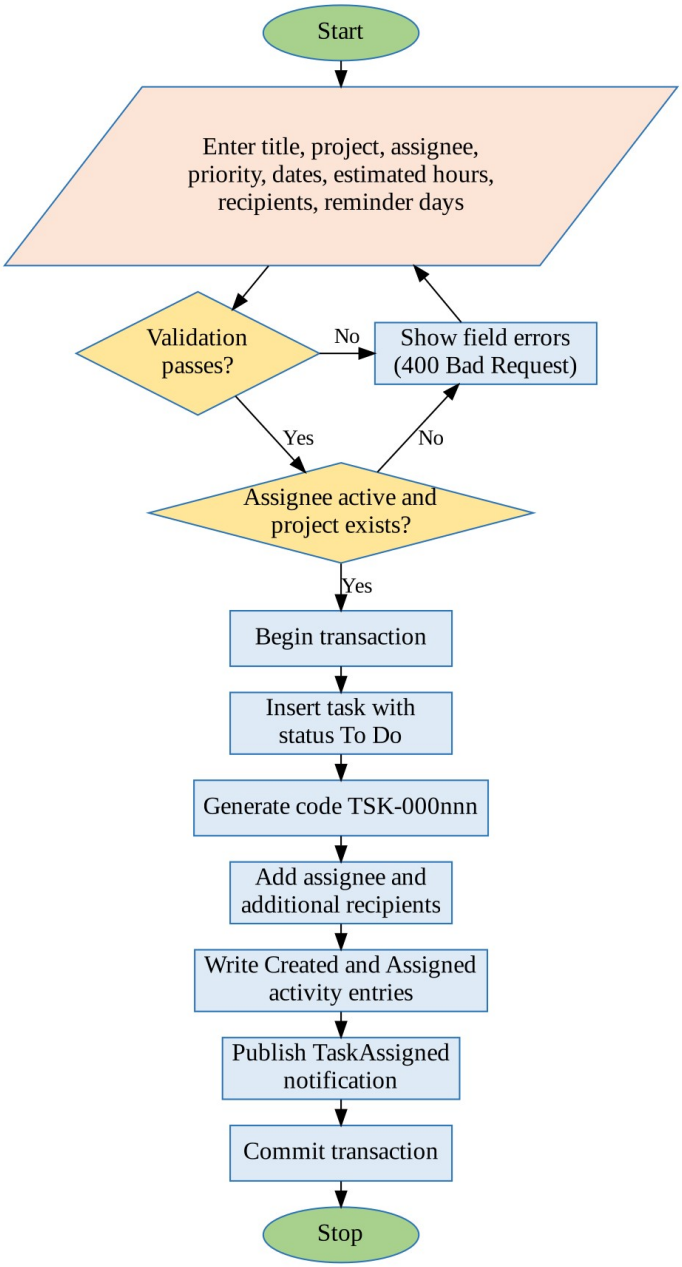


Figure 4.10 Flowchart For Task Creation

The rules by which the status of a task has been governed have been shown as a state transition diagram in Figure 4.11, in which every permitted move between two statuses has been drawn as a separate arrow between the corresponding states. A task has always been created in the To Do status, from which it was allowed to move to In Progress, Blocked or Cancelled, and from In Progress it was allowed to move to In QA, Blocked, Completed or Cancelled. From In QA the task was allowed to return to In Progress or to move to Blocked, Completed or Cancelled, and from Blocked it was allowed to move only to In Progress or Cancelled, while Completed and Cancelled have been treated as terminal states from which no further move was allowed. These rules have been implemented in a pure rule class named

TaskStatusTransitions, which has been tested directly by unit tests so that an illegal move could never have been accepted by the server.

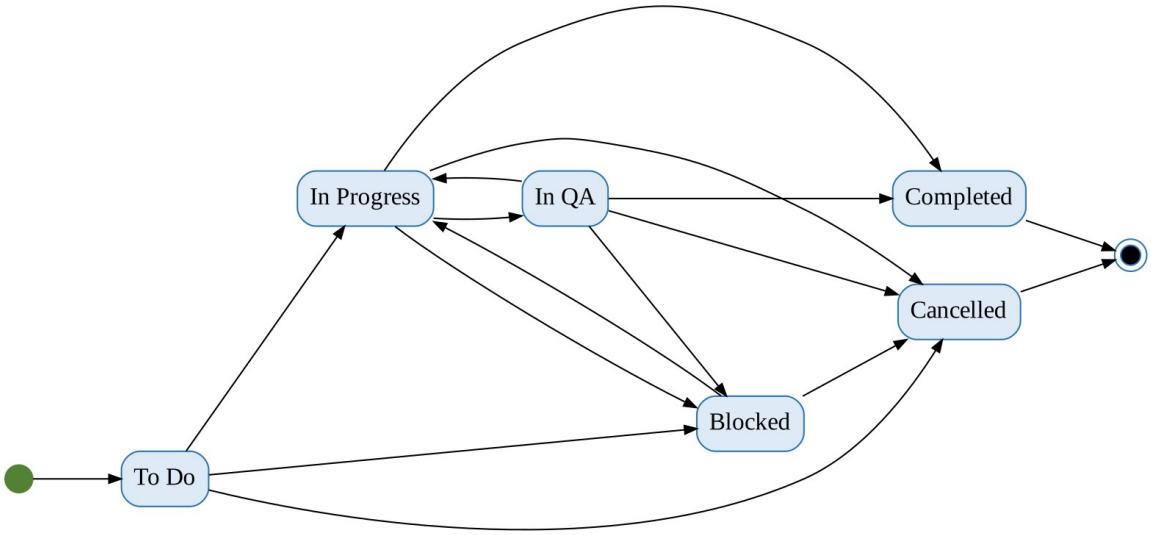


Figure 4.11 State Transition Diagram Of Task Status

The process through which the status of a task has been changed by a user has been shown as a flowchart in Figure 4.12, and this process has been followed both when a new status was selected on the task details screen and when a card was dragged on the Kanban board. The request has first been checked against the tasks.status.change permission, after which the access of the user to the particular task has been verified, and access has been granted only to the assignee, the creator, an additional recipient or a holder of the tasks.view.all permission. The requested move has then been checked against the transition table, a refused move has been answered with a clear message, and an accepted move has been saved together with the completion or cancellation time and a StatusChanged entry in the activity history.

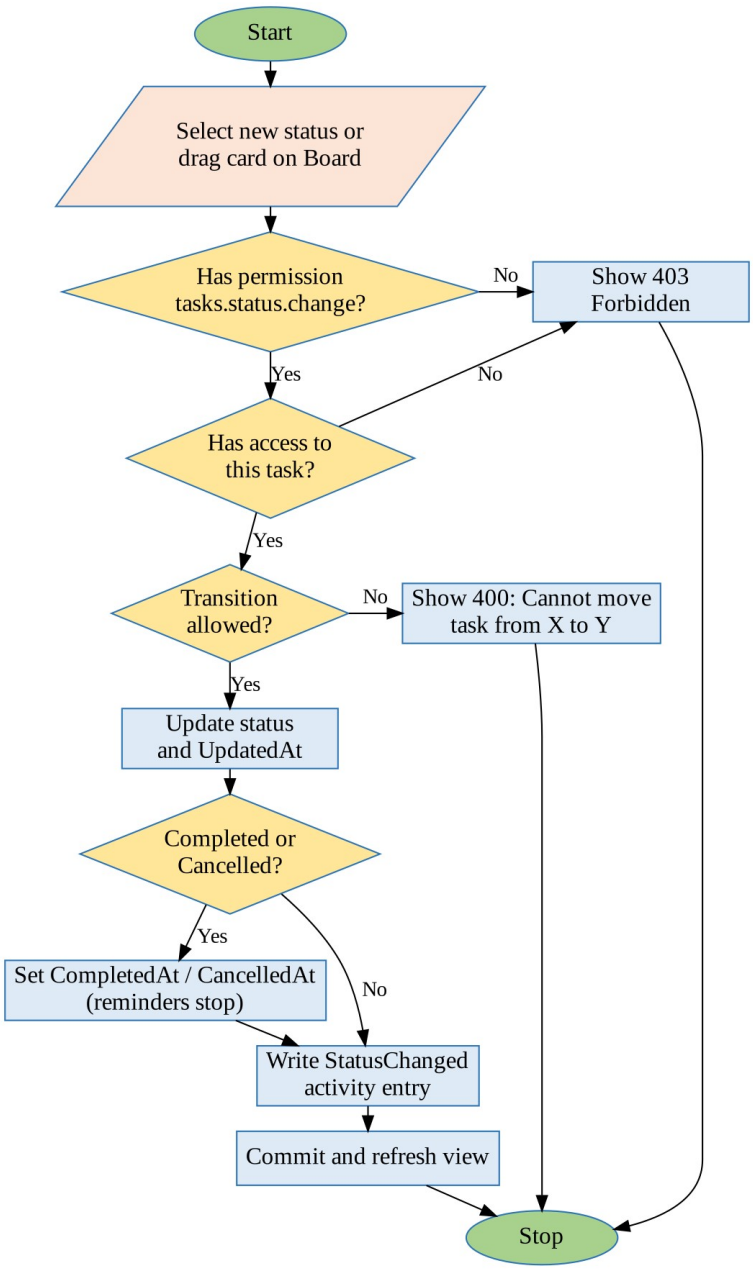


Figure 4.12 Flowchart For Task Status Change

The automatic reminder engine, which has been regarded as the central intelligent feature of TaskFlow, has been shown as a flowchart in Figure 4.13 with all its decisions. On every tick of the scheduler it was checked whether reminders had been enabled in the settings, all the open tasks that were due within the next thirty-two days or that were already overdue were read, and it was decided for each task whether a first reminder, a second reminder, a due-today reminder or an overdue reminder was due on the current day in the application time zone. For every recipient, consisting of the assignee, the additional recipients and the creator, a reminder history row has been inserted with an ON CONFLICT DO NOTHING clause, and only when a new row had actually been inserted were the in-app notification and, if enabled, the

system mail created inside the same transaction, so that each reminder has been delivered exactly once.

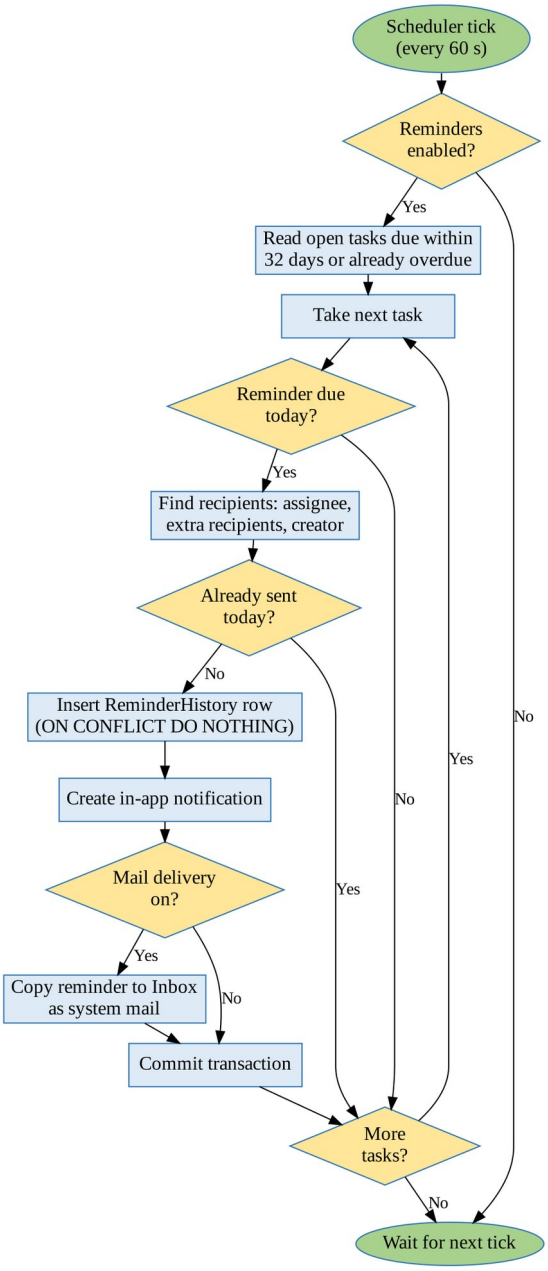


Figure 4.13 Flowchart For The Automatic Reminder Engine

The processing of a message by the TFA assistant has been shown as a flowchart in Figure 4.14, in which the safety checks, the document search and the confirmation step have been represented. The message has first been screened so that anything which looked like a password or an API key was refused before it was sent to any external model, and a rate limit of twenty requests per minute has been applied to each user. The most relevant sections of the project documents have then been retrieved by a hybrid search in which vector similarity was combined with an SQLite FTS5 keyword search through reciprocal rank fusion, and the prompt, the conversation history and the retrieved context were sent to the selected AI provider.

When a read tool was requested by the model, the tool was executed under the permissions of the caller, and when an action such as sending a mail was requested, a pending action was created that was executed only after the user had confirmed it, and every outcome has been written to an audit table.

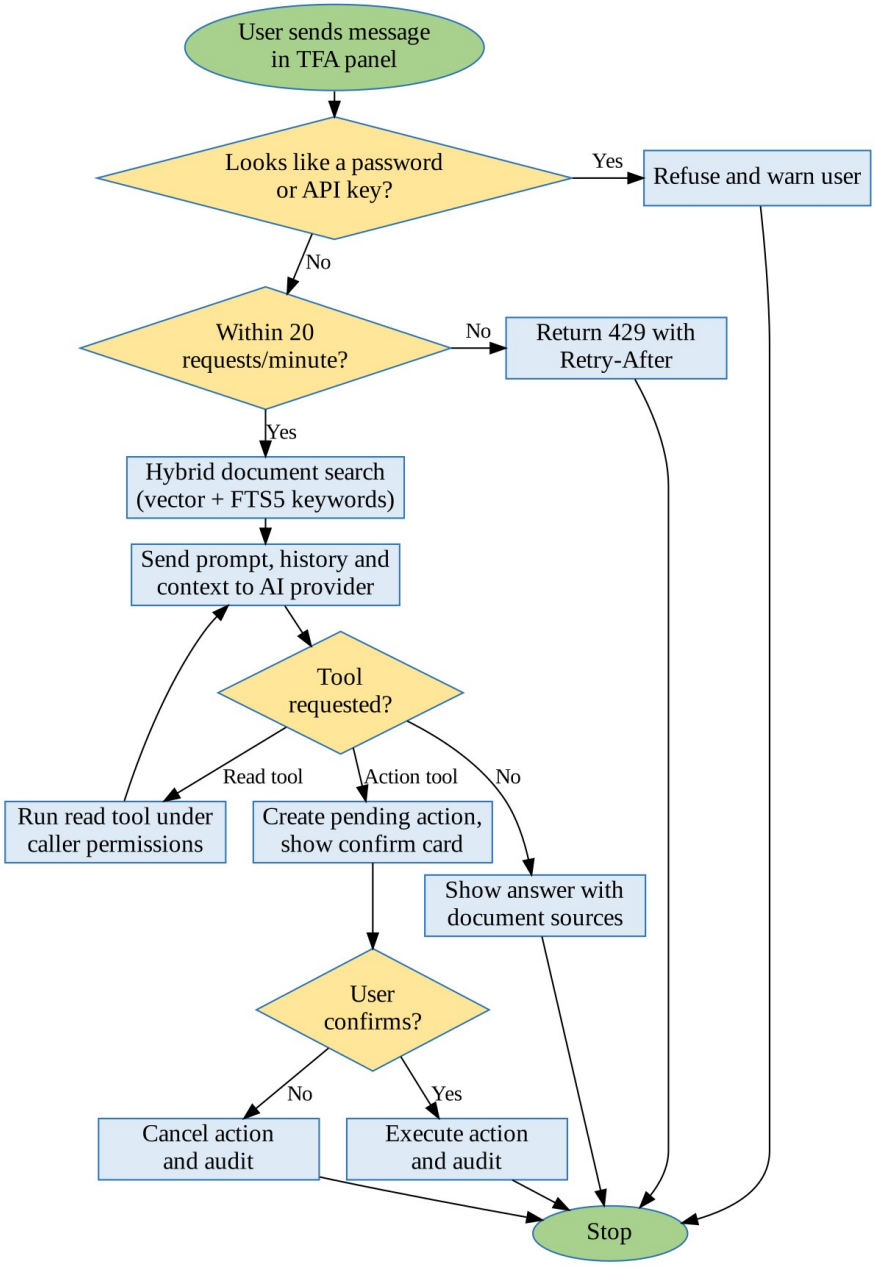


Figure 4.14 Flowchart For TFA Assistant Message Processing

4.1.2. Concept Design

The concept design has identified the main concepts of the problem domain and the rules by which they were related, independently of any particular screen or table of the system. The task has been treated as the central concept of TaskFlow, which belonged to a project, was assigned to exactly one employee at a time, was created by one user and could have any number

of additional recipients who were informed about it. A priority of Low, Medium, High or Critical, a start date, a deadline, estimated and actual hours, a status and three reminder settings have been given to each task, and comments and activity entries have been collected for it over its life. Overdue has been treated as a derived concept, which was true when the deadline had passed and the task was neither completed nor cancelled, and the display stage of Upcoming, Due Today, Overdue or Completed has likewise been computed whenever the task was read.

The second important concept has been access control, which has been designed around permissions rather than fixed role names so that access could be changed without any change to the code. A permission has been defined as a dotted key such as tasks.create or reports.export, and a fixed catalogue of seventy such keys that were grouped by module has been included in the system. A role has been defined as a named set of permissions, and exactly one role has been held by every user, which was either one of the three read-only system roles or a custom role that had been created by an administrator. A data scope of Own, Team or Organisation has also been selected by some permissions, by which it was decided whether only the personal data, the data of the team or the data of the whole organisation was shown to a user, and the default access of the three system roles has been summarised in Table 4.1.

Table 4.1 Default Access Of The System Roles

Module	Administrator	Manager	Employee
Dashboard	Organisation scope	Team scope	Own scope
Task Monitoring	All operations	All except delete	Own tasks, status, hours, comments
Board and Workload	Full access	Full access	Board only
Employee, Project, Department	Full management	View only	No access
Notifications and Mail	Own mailbox	Own mailbox	Own mailbox
Reports	View and export	View and export	No access
Settings and Roles	All settings	Reminder settings	No access
My Profile	Always on	Always on	Always on
TFA Assistant	Chat and confirm	Chat and confirm	Chat and confirm

The third concept has been the reminder, which has been defined as a message that was generated automatically for a task on a particular day according to the reminder intervals of the task. A first reminder interval, a second reminder interval and an overdue frequency have been copied into each task from the global reminder settings when it was created, and the default values have been set to two days, one day and daily respectively. A reminder has been identified by the task, the recipient, the reminder type and the scheduled day, and this

combination has been kept unique, which has formed the conceptual basis of the guarantee that no reminder was ever delivered twice to the same person. The four types of reminder, the conditions under which they were generated and the messages that were produced have been listed in Table 4.2.

Table 4.2 Reminder Types And Conditions

Reminder Type	Condition (Application Time Zone)	Message Produced
FirstReminder	Days until deadline equal the first reminder interval (default 2)	Reminder: task is due in n day(s)
SecondReminder	Days until deadline equal the second reminder interval (default 1)	Reminder: task is due in n day(s)
DueToday	Deadline is today and the deadline time has not yet passed	Reminder: task is due today at HH:mm
Overdue	Deadline has passed; every day (Daily) or every 7th day (Weekly)	Overdue: task was due on date
None	Task is Completed, Cancelled or deleted	No reminder is generated

The remaining concepts of the system have been the internal mail, the notification and the assistant conversation, each of which has been modelled on a familiar idea from everyday software. The internal mail has been modelled on the familiar e-mail concept, in which a message had one sender, To and Cc recipients, a subject, a body and attachments, and a personal copy of the message was received by each user in a folder such as Inbox, Sent or Drafts with personal flags for read, starred, important and trash. A notification has been defined as a short message for one user about an event such as a task assignment, a reassignment, a comment or a reminder, which could be marked as read by the user. A conversation with the assistant has been made up of user, assistant and tool messages, and an action that had been proposed by the assistant has been stored as a pending action that expired if it was not confirmed in time.

4.1.3. Logical Design

The logical design has described the software structure of the system in terms of layers, components and interfaces, which have been derived directly from the folder structure of the code. The back end has been organised as a single ASP.NET Core project with a layer-first folder structure, in which sub-folders for the modules were kept inside each layer, and each layer was allowed to call only the layer immediately below it through interfaces. Only HTTP routing, binding and status codes have been handled by the controllers, the business rules and the use cases have been placed in the services, and the Entity Framework Core queries have been kept in the repositories without the changes ever being saved there, because all the

changes were committed together by the services through a unit of work. These rules have been enforced automatically by architecture tests that were run with every build, and the responsibilities of the layers have been summarised in Table 4.3.

Table 4.3 Layers Of The Back End

Layer (Folder)	Responsibility	May Call
Controllers	HTTP routing, model binding and status codes	Services
Services	Business rules, validation of state, use cases	Repositories, Integrations
Repositories	EF Core queries and changes (no saving)	Persistence
Integrations	JWT, TOTP, Data Protection, file storage, AI providers	External systems
Dtos and Models	Request and response records, validators, entities, enums	Nothing
Middleware and Filters	Exception handling, logging, session check, MFA gate, validation	Services
Persistence	AppDbContext, migrations and seeders	SQLite database

The interface between the front end and the back end has been designed as a REST API under the base path /api/v1, in which JSON property names were written in camel case, enumerations were sent as their names, the items were returned by list endpoints together with the total, page and page size, and every error was returned as an RFC 7807 ProblemDetails document with a trace identifier. One hundred and fifteen actions in twenty-one controllers have been included in the API, and every action has been protected by an explicit permission policy, by the resource based task access check, or by an explicit anonymous attribute in the case of the sign-in and recovery endpoints. The distribution of the actions over the modules, together with examples of the endpoints that were exposed by each module, has been shown in Table 4.4.

Table 4.4 Summary Of The REST API

Module	Controllers	Actions	Example Endpoints
Authentication	AuthController, PasswordRecoveryController	7	POST /auth/login, POST /auth/mfa/verify
Dashboard	DashboardController, AnalyticsController	8	GET /dashboard/admin/analytics
Management	Employees, Departments, Projects, Positions	26	GET /users, POST /projects
Monitoring	Tasks, TaskComments, TaskActivity, Workload	20	POST /tasks, PATCH /tasks/{id}/status
Communication	MailController, NotificationsController	20	POST /mail/send, GET /notifications
Reports	ReportController	2	GET /reports/export
Profile	ProfileController	7	POST /users/me/password
Settings	ApplicationSettings, ReminderSettings, Roles, AiProviders	18	PUT /settings/reminders
Assistant	AssistantController	7	POST /assistant/conversation/messages
Total	**21 controllers**	**115**	

The front end has been designed logically as a shell with lazy loaded feature modules, so that only the code that was needed for the current screen was downloaded by the browser. The router, the HTTP client with the API base URL interceptor and the authentication interceptor, and an initialiser by which the session was restored when the page was loaded have been registered in the application configuration. All the authenticated screens have been made children of the shell component, by which the header, the side menu that was built from the permissions of the user, the notification bell, the legal footer and the TFA assistant panel were displayed, and which was protected by the authentication and two-step verification guards. The pages, components, services, models and utilities of each feature module have been kept in separate folders, and these placement rules have been verified by a structure checking script that was run before the front end tests.

4.1.4. Physical Design

The physical design has described how the system was installed on the hardware and where its data was physically stored on the server. TaskFlow has been run as a single ASP.NET Core process in which the REST API, the reminder scheduler and the document indexer were hosted, and in production the compiled Angular application has been served as static files by a web server or a static hosting service, while a Dockerfile has also been provided for container

based deployment. All the local state of the system has been kept in one data folder, which was C:\TaskFlow on Windows and a TaskFlow folder under the common application data folder on other operating systems, and the location could be changed through the Storage:Root configuration key. The contents of the data folder and the other physical components of the system, together with the purpose for which each of them has been used, have been listed in Table 4.5.

Table 4.5 Physical Components And Storage

Component	Physical Location And Purpose
taskflow.db	SQLite database file in the data folder holding all 33 tables
dp-keys	Data Protection key ring that encrypts MFA secrets and AI API keys (DPAPI on Windows)
mail-attachments	Uploaded mail attachments stored under generated GUID file names
appsettings.json	Non-secret defaults such as token lifetimes, time zone and scheduler tick
User secrets / environment	Secrets such as the JWT signing key and the Gemini API key
TaskFlow.Api process	Web API on https://localhost:7055 with the hosted background services
Angular application	Development server on http://localhost:4200; dist folder in production
TF - Documents folder	Markdown design documents indexed for the TFA assistant

The configuration of the system has been separated physically into three levels so that secrets were never stored in the source code or in the database of the system. Non-secret defaults have been kept in the appsettings.json file, per-environment values such as the allowed CORS origins and the base address of the application have been kept in an environment file, and secrets such as the JWT signing key and the Gemini API key have been supplied through the .NET user secrets store or through environment variables. Settings that the administrator had been allowed to change at run time, such as the application name, the time zone, the default priority, the reminder defaults and the notification switches, have been stored as key and value rows in the ApplicationSettings table and have been edited through the settings screens. A backup of the system has been defined simply as a copy of the data folder that was taken while the API was stopped, so that the whole system could be restored by copying the folder back.

4.1.5. Input Design

Input design has been defined as the process through which the information that has been supplied by the users was converted into a form that could be processed by the computer, and its aim has been to make the entry of data easy, quick and free of errors. In TaskFlow, every

input screen has been designed with clearly labelled fields, sensible default values, drop-down lists for values that have been taken from master data, date and time pickers, toggles for yes or no settings and inline messages that have been placed next to the field concerned. Validation has been performed twice, first in the browser for the convenience of the user and then on the server through FluentValidation validators, which have been treated as authoritative, so that invalid data could never be stored even when the browser checks had been bypassed. The main input forms of the system have been described in this sub-section together with their fields and the validation rules that have been applied to them.

The sign-in screen has been designed as the first input screen that a user encounters, and the fields that have been placed on it have been listed in Table 4.6. The user has been allowed to enter either the email address or the user name together with the password, and after the password had been accepted, the same screen has been made to ask for the six-digit code from the authenticator application or for one of the backup codes. When two-step verification had not yet been set up by the user, a QR code and a secret key have been shown on the Security Setup screen, and these have been scanned with an authenticator application such as Google Authenticator or Microsoft Authenticator, after which the first code has been entered to enable the protection. Only internal paths have been accepted as the return address after sign-in, so that a user could never have been redirected to an external site by a crafted link.

Table 4.6 Input Fields Of The Sign-In Screen

Field	Control	Validation Rule
Email or Username	Text box	Required; matched against the user name or the lower-case email
Password	Password box	Required; 6 to 128 characters; never stored or logged in plain text
Authenticator Code	Numeric text box	Six digits; current 30-second step plus or minus one; each step used once
Backup Code	Text box	Format XXXXX-XXXXX; single use; accepted instead of the authenticator code
Remember Return URL	Hidden	Only internal paths are accepted for redirection after sign-in

The appearance of the sign-in screen, which has been the only screen that has been shown to a visitor who had not yet signed in apart from the account recovery and legal pages, has been shown in Figure 4.15. The screen has been designed with a field for the email address or user name, a field for the password, a sign-in button and a link to the account recovery screen, and after the password had been accepted, the same card has been changed to request the six-digit authenticator code or a backup code. Every failure of the sign-in has been reported with the same general message, so that the existence of an account could never have been

discovered by an outsider, and the links to the legal documents have been placed in the footer of the screen.

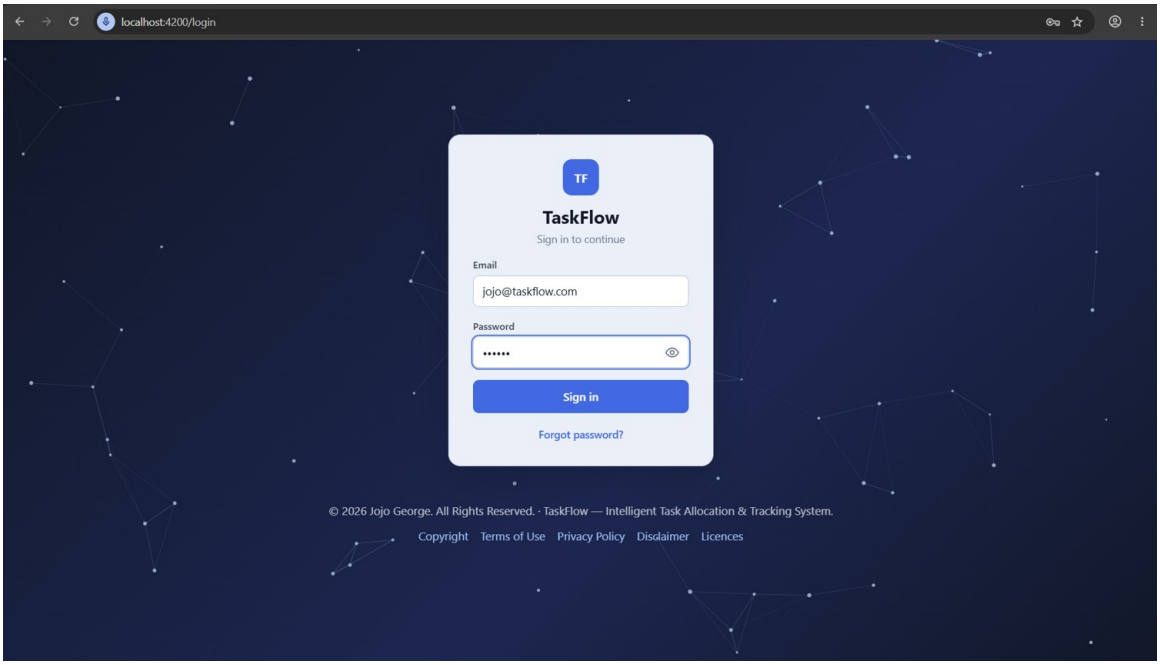


Figure 4.15 Sign-In Screen

The Security Setup screen, through which two-step verification has been enabled by every user on the first sign-in, has been shown in Figure 4.16. A twenty-byte secret has been generated by the server after the current password had been confirmed, and this secret has been shown both as a QR code and as a Base32 key, so that it could be added to an authenticator application by scanning the code or by typing the key. The first six-digit code that had been produced by the application has then been entered to prove that the setup had succeeded, after which ten single-use backup codes in the form XXXXX-XXXXX have been displayed once, and the user has been advised to keep them in a safe place for the case in which the phone had been lost.

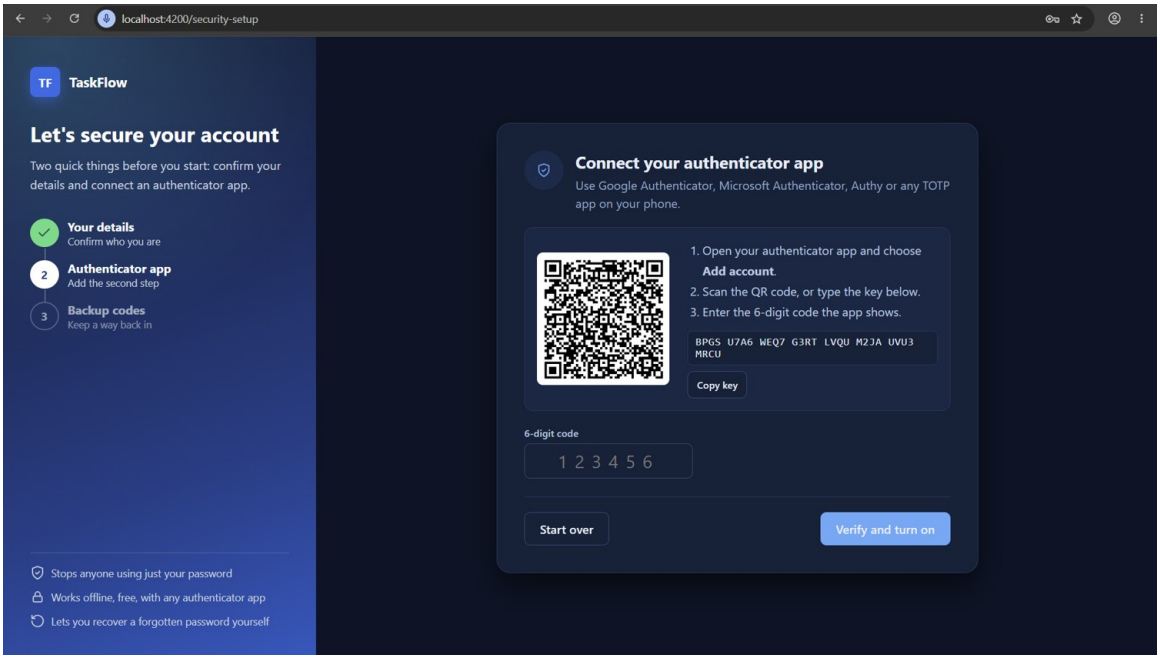


Figure 4.16 Security Setup Screen With QR Code

The Create Task screen has been treated as the most important input screen of the system, and the fields that have been provided on it have been listed in Table 4.7. The screen has been made available only to users who have been granted the tasks.create permission, and only the active employees have been included in the list of assignees, so that a task could never be assigned to a person who had left the organisation. The priority and the three reminder fields have been filled in automatically with the defaults that had been configured in the application and reminder settings, and these values could have been changed by the user before saving. When the task has been saved, the task code has been generated by the server and the user has been returned to the task details, and whenever any rule had been broken, the corresponding message has been shown next to the field.

Table 4.7 Input Fields Of The Create Task Screen

Field	Control	Validation Rule
Title	Text box	Required; maximum 200 characters
Description	Multi-line text	Optional; maximum 4000 characters
Project	Drop-down list	Required; must be an existing active project
Assigned To	Drop-down list	Required; must be a live and active employee
Priority	Drop-down list	Low, Medium, High or Critical; default from settings
Start Date	Date and time picker	Required
Deadline	Date and time picker	Required; must be after the start date
Estimated Hours	Numeric box	Required; must be greater than zero
Additional Recipients	Multi-select list	Optional; at most 50 live users
First Reminder Days	Numeric box	0 to 30; default 2 from reminder settings
Second Reminder Days	Numeric box	0 to 30; not before the first reminder; default 1
Overdue Frequency	Drop-down list	Daily or Weekly; default Daily

The Create Task screen, through which all the details of a new task have been entered by a manager or by any other user who had been granted the tasks.create permission, has been shown in Figure 4.17. The fields of the screen have been arranged in the order in which they have been listed in the input table, with the title and the description at the top, followed by the project, the assignee, the priority, the start date, the deadline and the estimated hours. The additional recipients and the three reminder settings have been placed in the lower part of the form, and the save button has been enabled only after all the mandatory fields had been filled, while the server has repeated every validation before the task was stored.

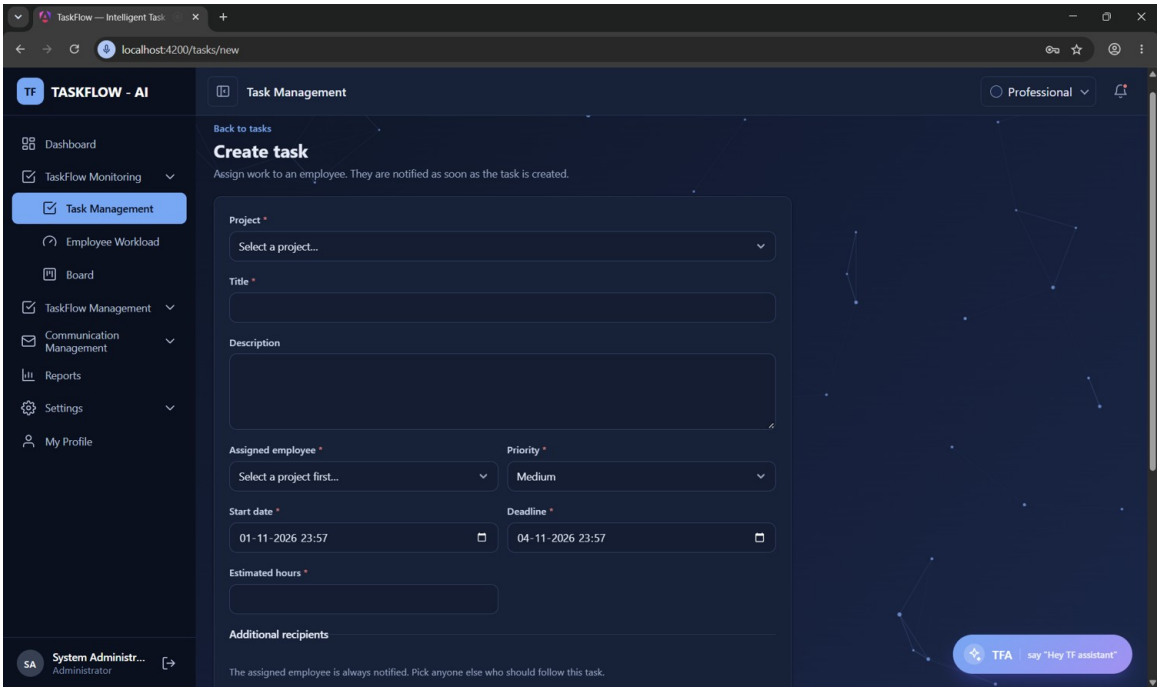


Figure 4.17 Create Task Screen

The Employee Management screen has been used by the administrator to create and maintain the accounts of the employees, and the fields of the employee form have been listed in Table 4.8. Every email address has been placed on the fixed taskflow.com domain and has been kept unique across all users, including the users who have been deleted, so that an address could never be reused accidentally, and the user name has also been kept unique. One role has been assigned to each employee by the administrator, the employee has been allocated to one or more projects with a position and an allocation percentage whose total could not exceed one hundred, and the employee has been added to one or more departments. A deactivated employee has been prevented from signing in, and every live session of that employee has been ended on the next request, because the security stamp of the account has been changed.

Table 4.8 Input Fields Of The Employee Form

Field	Control	Validation Rule
Employee Code	Text box	Optional; maximum 32 characters
Full Name	Text box	Required; maximum 200 characters
Username	Text box	Required; maximum 100 characters; unique
Email	Text box	Required; on the @taskflow.com domain; unique including deleted users
Mobile Number	Text box	Optional; E.164 format such as +919876543210
Password	Password box	Required on creation; 6 to 128 characters
Role	Drop-down list	Required; system or custom role the caller may assign
Status	Toggle	Active or Inactive; inactive users cannot sign in
Project Allocation	Repeating rows	Project, position and percentage 1 to 100; total at most 100
Departments	Multi-select list	Optional; existing departments

The Employee Management screen together with the employee form, as it has appeared on the screen of the administrator during the creation of a new account, has been shown in Figure 4.18. The upper part of the screen has contained a searchable list of the employees with their code, name, email, role, departments and status, and the form has been opened in a panel when a new employee was added or an existing employee was edited. The project allocations have been entered as repeating rows in which a project, a position and a percentage were chosen, and the total percentage has been shown while the rows were being edited, so that an allocation of more than one hundred percent could be noticed and corrected before the form was saved.

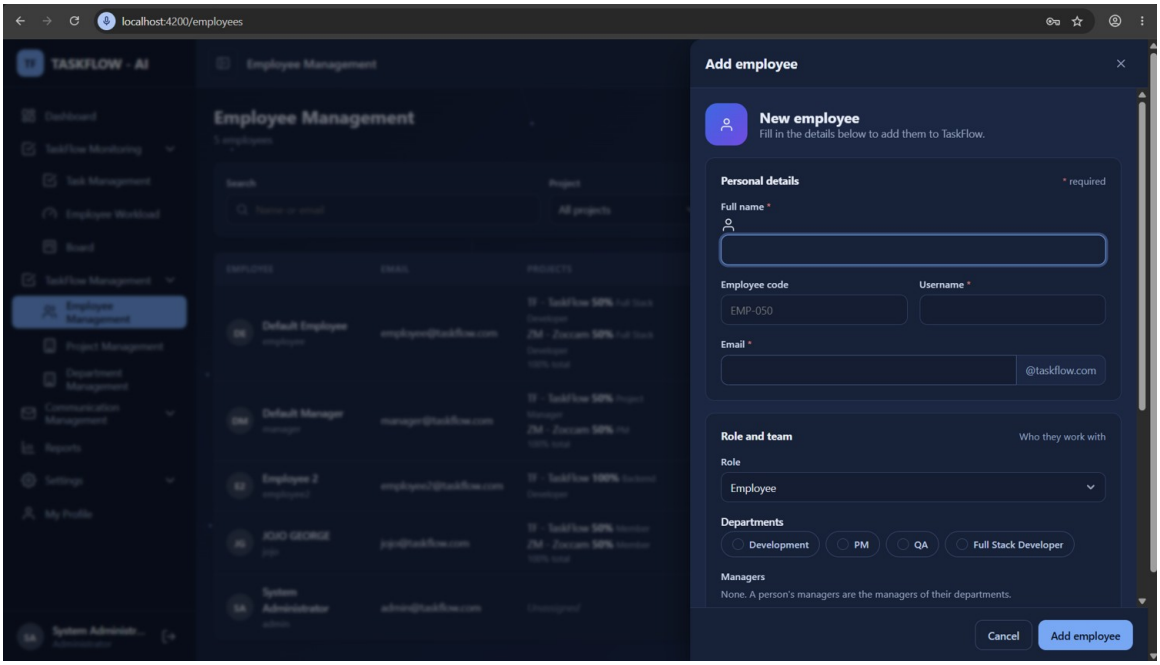


Figure 4.18 Employee Management Screen With Employee Form

The Deadline Reminder Settings screen has been provided so that the behaviour of the reminder engine could be changed by an authorised user without any change to the program code, and the fields of this screen have been listed in Table 4.9. The master switch has been used to turn the whole engine on or off, the deadline-day switch has been used to control the due-today reminder, and the three default values have decided the intervals that have been copied into every new task. The switch for delivering reminders to mail has decided whether a copy of each reminder has also been placed in the internal mail box of the recipient, and the time zone, which has been owned by the application settings screen, has been shown only for information. All six editable fields have been made mandatory on saving, and missing or unreadable values have been replaced by the defaults that have been defined in the code.

Table 4.9 Input Fields Of The Reminder Settings Screen

Field	Control	Validation Rule
Send Deadline Reminders	Toggle	Required; default On
Deadline Day Reminder	Toggle	Required; default On
Default First Reminder (days)	Numeric box	1 to 30; default 2
Default Second Reminder (days)	Numeric box	0 to 30 and less than the first; default 1
Default Overdue Frequency	Drop-down list	Daily or Weekly; default Daily
Also Deliver Reminders To Mail	Toggle	Required; default On
Application Time Zone	Read-only text	Shown from application settings (Asia/Kolkata)

The Deadline Reminder Settings screen, through which the behaviour of the automatic reminder engine has been configured by an authorised user, has been shown in Figure 4.19. The two switches for sending deadline reminders and for the deadline-day reminder have been placed at the top of the screen, and the default first reminder days, the default second reminder days and the default overdue frequency have been placed below them, together with the switch that has decided whether every reminder was also delivered to the internal mail box. The application time zone has been displayed on this screen only for information, because it has been owned by the application settings screen, and a confirmation message has been shown after the settings had been saved successfully.

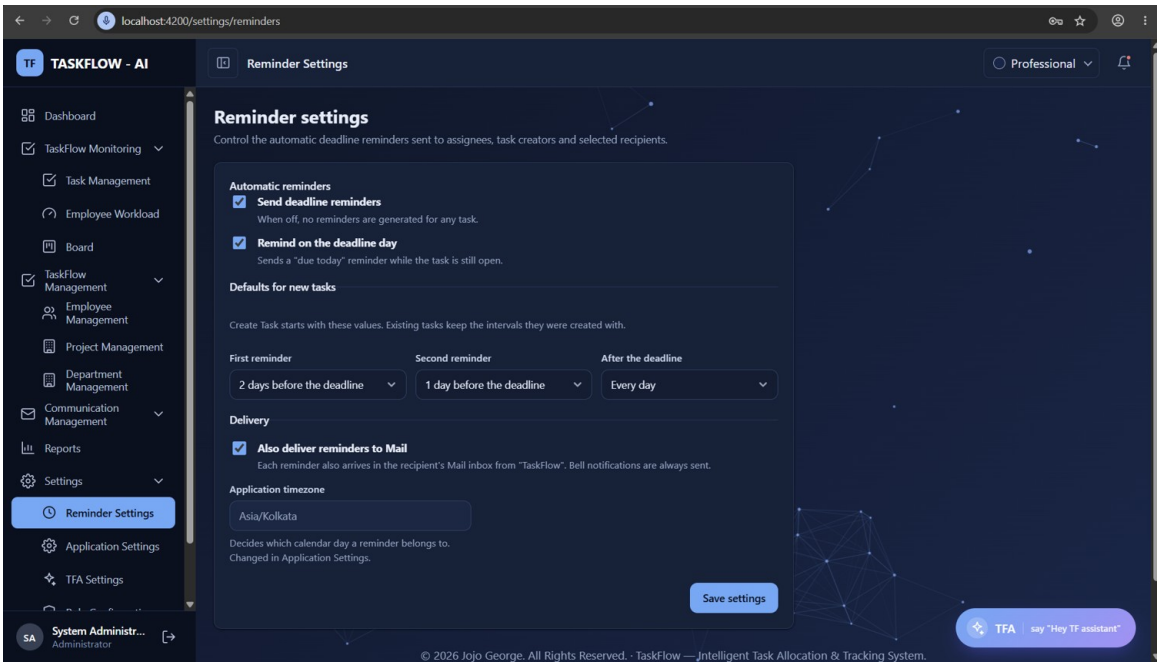


Figure 4.19 Deadline Reminder Settings Screen

The internal mail compose screen, which has been another frequently used input screen of the system, has been shown in Figure 4.20. The To and Cc fields have been filled with TaskFlow users through a contact search that has returned at most ten active users matching the typed name or email, and a subject of up to two hundred characters and a plain text body have been entered below them. Attachments have been uploaded one at a time with a request limit of eleven megabytes, and files with blocked extensions have been refused, while a message that had not yet been sent has been saved as a draft that could be reopened later from the Drafts folder and completed before it was finally sent.

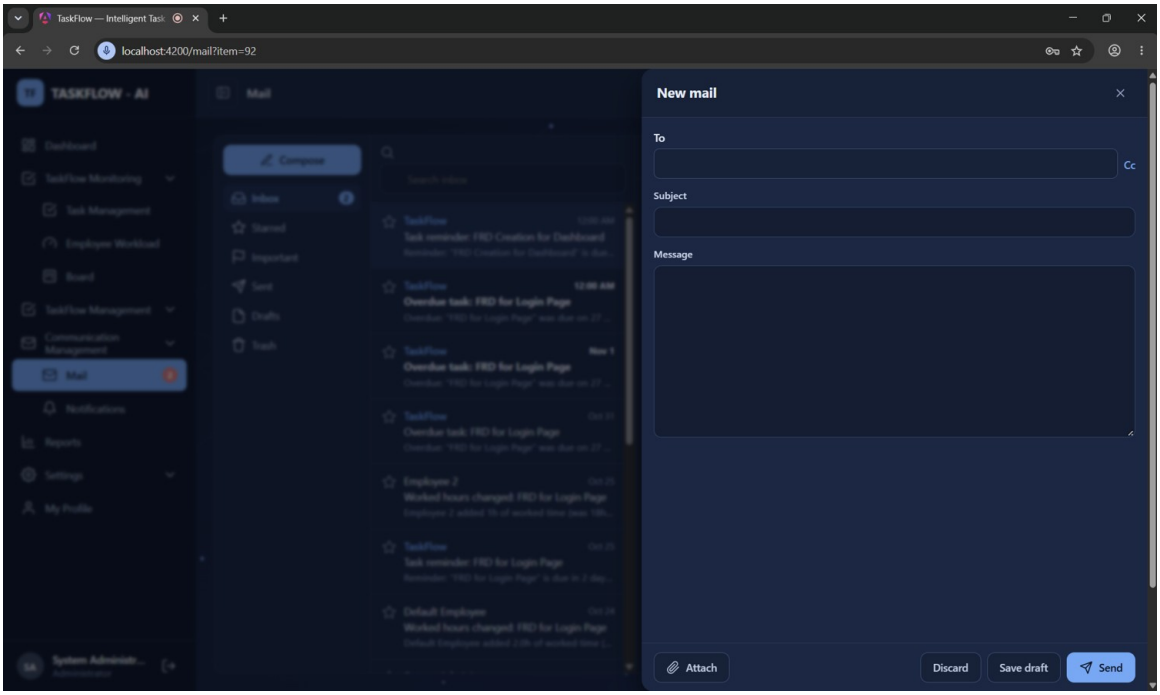


Figure 4.20 Internal Mail Compose Screen

4.1.6. Output Design

Output design has been concerned with the presentation of the processed information to the users in a form that was accurate, timely and easy to understand, because the outputs have been regarded as the primary means through which the usefulness of a system had been judged by its users. The outputs of TaskFlow have mainly been on-screen outputs such as dashboards with key figures and charts, paged and filtered lists, the Kanban board, detailed task views and notification lists, together with the CSV file that has been produced by the reports module and the reminder messages that have been delivered through notifications and mail. Every output has been restricted to the data that the user has been permitted to see, the dates have been shown in the application time zone, and overdue tasks have been highlighted consistently on every screen so that they have attracted immediate attention.

The dashboards have been the first outputs that a user has seen after signing in, and three versions of them have been designed for the three scopes of data, namely Organisation, Team and Own. The administrator dashboard in Figure 4.21 has been designed to show organisation wide figures such as the total, pending, in progress, completed and overdue tasks, together with charts of tasks by status, priority, project and department and filters for the period, department, project, manager and employee. The analytics of this dashboard have been computed by the server from the tasks of the whole organisation, and the default date range, the limits and the application time zone have been applied by a filter resolver before the figures were calculated. Every figure on the dashboards has been made clickable so that the matching list of tasks could be opened, which has made it easy to move from a summary to the details.

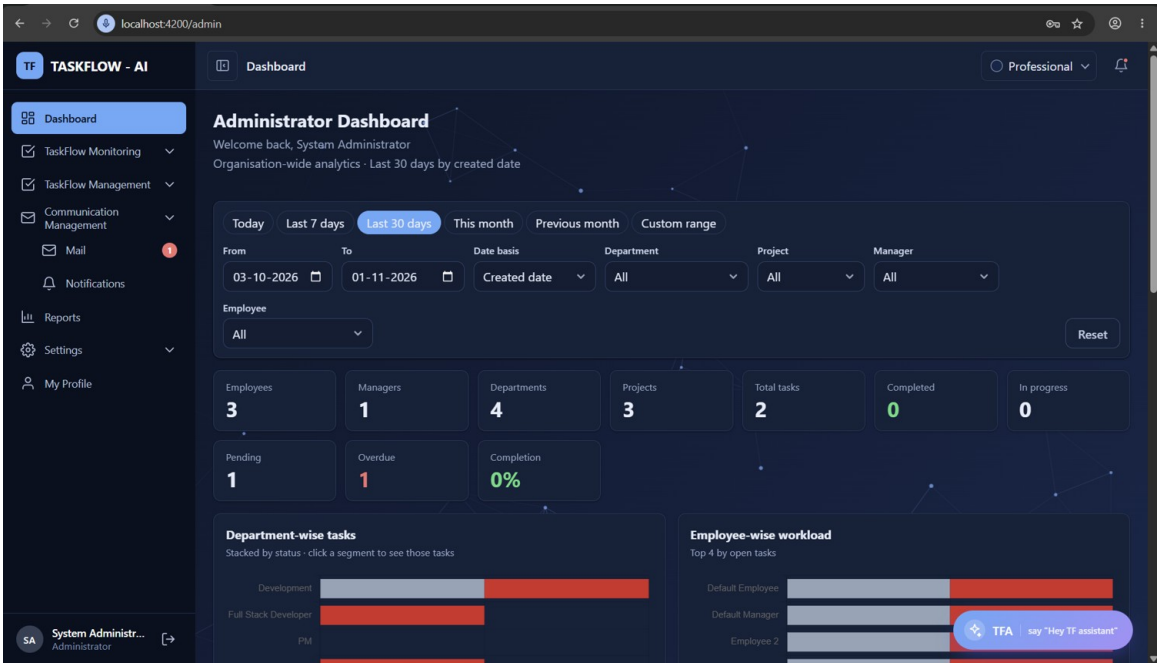


Figure 4.21 Administrator Dashboard

The manager dashboard, which has been shown in Figure 4.22, has presented the same kind of information as the administrator dashboard, but the figures and charts have been limited to the team of the signed-in manager. The team scope has been enforced by the server, so that a request for the analytics of another team has been refused with the 403 status even when the request had been crafted outside the browser, and this behaviour has been verified by dedicated isolation tests. The filters of this dashboard have offered only the departments, projects and employees that had belonged to the team of the manager, and every count on the dashboard has been linked to the matching list of tasks, so that delayed work could be investigated immediately.

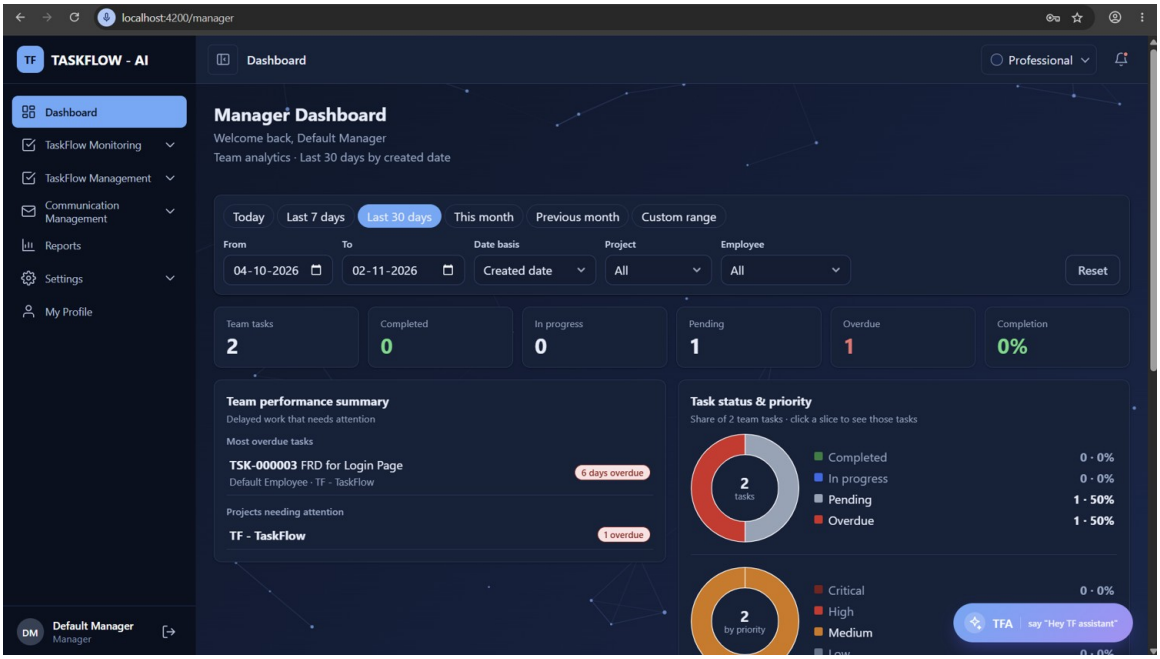


Figure 4.22 Manager Dashboard

The employee dashboard, which has been shown in Figure 4.23, has presented the personal figures of the signed-in employee, including the number of tasks that had been assigned, the tasks that have been due today, the tasks that had become overdue and the tasks that had been completed. This dashboard has been made available to every signed-in user, because the permission to view the own dashboard has been treated as an always-on permission that could not be removed from any role. The analytics of this dashboard have accepted only a period as input, and any other filter that had been sent by the browser has been ignored by the server, so that the data of other employees could never have been revealed through this screen.

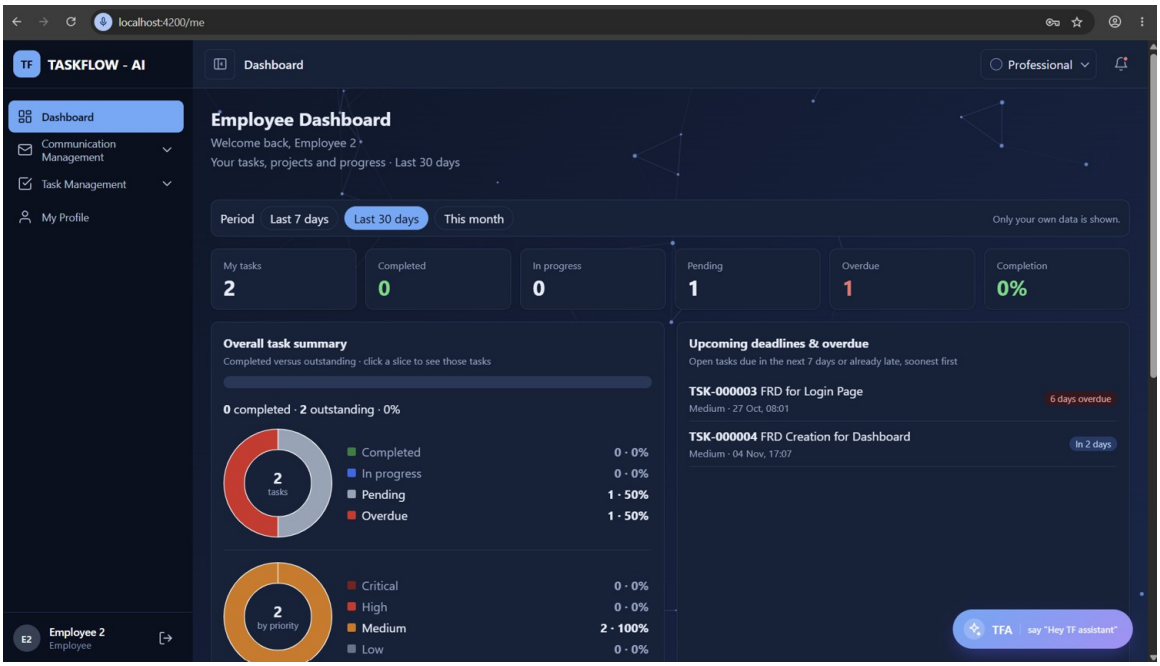


Figure 4.23 Employee Dashboard

The task list, which has been shown in Figure 4.24, has displayed the tasks that the user has been allowed to see in a paged table with the task code, title, project, assignee, priority, status, deadline and stage, and it has been made searchable and filterable. The list has been returned by the server twenty rows at a time by default, with a maximum of one hundred rows per page, so that the screen has remained fast even when a large number of tasks had been recorded. The scope of the list has been decided by the permissions of the user, so that an employee has seen only the tasks that had been assigned to, created by or shared with that employee, while a manager has seen the tasks of the team and an administrator has seen every live task of the organisation.

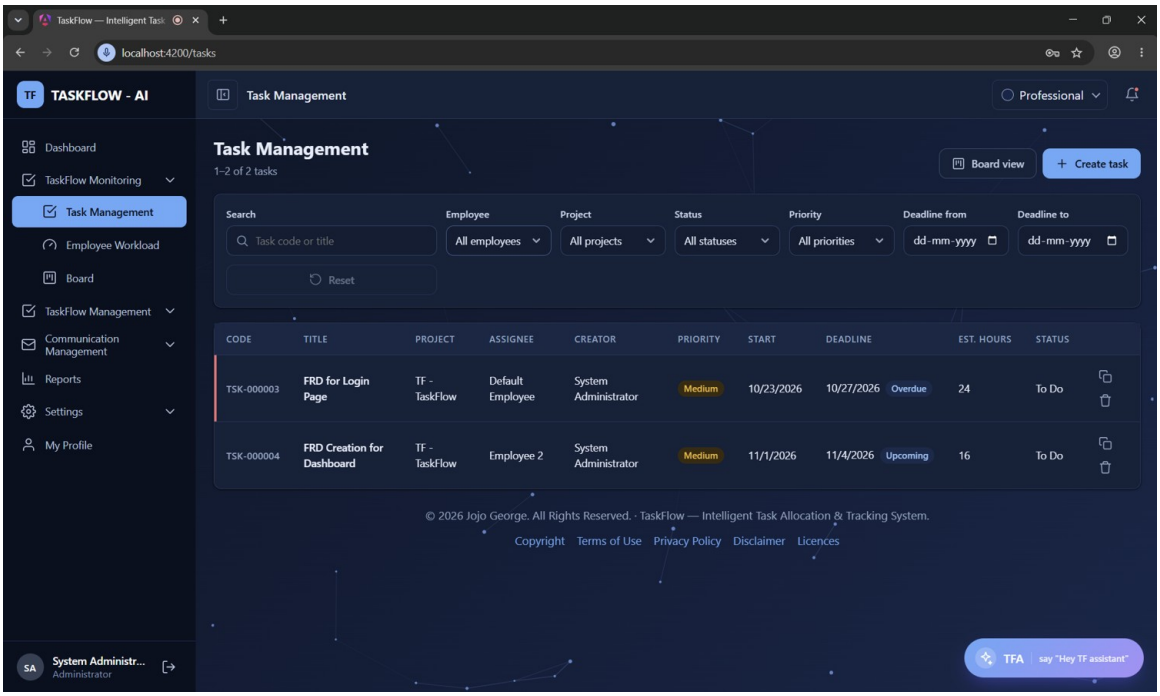


Figure 4.24 Task List Screen

The task details screen, which has been shown in Figure 4.25, has been designed to show all the information about one task, including the code, title, description, project, assignee, creator, priority, dates, recipients and the estimated and actual hours. Only the status changes that had been permitted by the transition rules for the current status have been offered on this screen, and the worked hours could be logged in entries of more than zero and at most twenty-four hours by the assignee or by a user who had been granted the tasks.update permission. Links to the discussion, the activity history and the reassignment screen have also been provided, and each of these links has been shown only to the users who had been granted the corresponding permission.

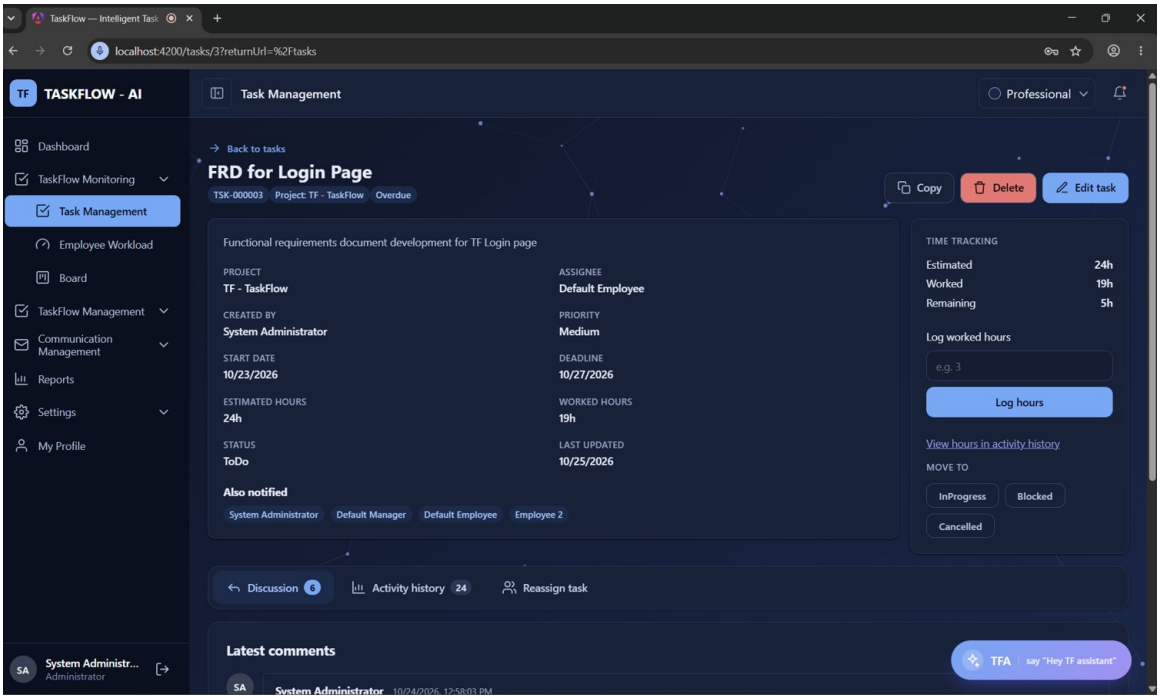


Figure 4.25 Task Details Screen

The Kanban board, which has been shown in Figure 4.26, has presented the tasks as cards in separate columns for the To Do, In Progress, In QA, Blocked, Completed and Cancelled statuses, and the board has been implemented with the drag-and-drop module of the Angular Component Dev Kit. A card could be dragged to another column only when the move had been permitted by the transition rules, and every accepted move has been sent to the same status endpoint that has been used by the task details screen. Each card has shown the task code, the title, the assignee, the priority and the deadline, and a card has been marked as overdue whenever its deadline had passed while the task was still open.

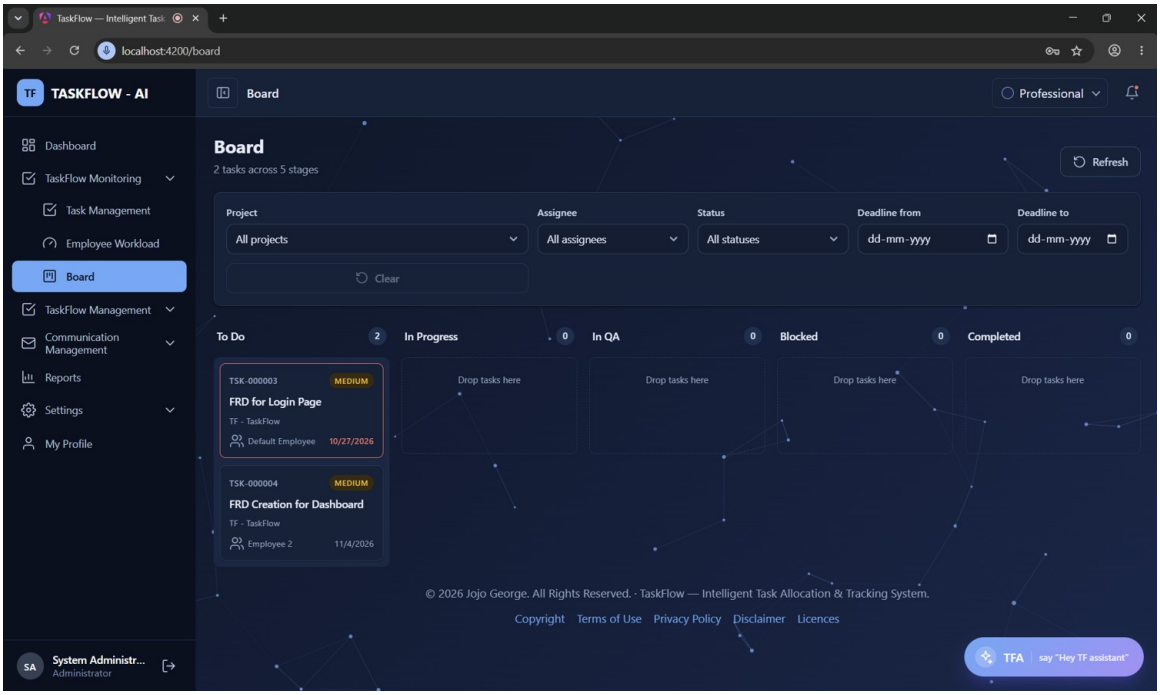


Figure 4.26 Kanban Board Screen

The employee workload screen, which has been shown in Figure 4.27, has presented the number of open, overdue and completed tasks and the total estimated hours for each employee, so that uneven distribution of work could be detected by managers at a glance. The figures of this screen have been aggregated by the workload service directly from the live tasks, and deleted tasks have been excluded from every count, so that the workload has always reflected the current state of the work. The screen has been protected by the workload.view permission, which has been granted to administrators and managers by default, and the high-load employees have been highlighted so that new tasks could be assigned to the people who had more free capacity.

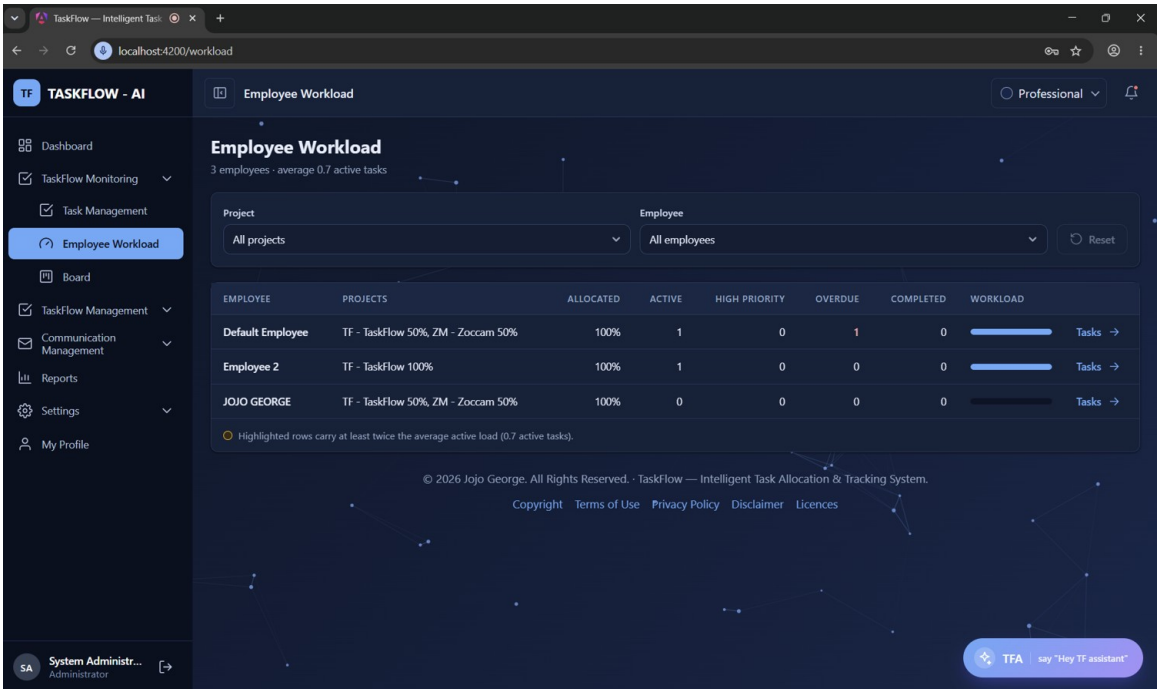


Figure 4.27 Employee Workload Screen

The notification centre in Figure 4.28 has listed the notifications of the user with the newest first, has distinguished the unread items, and has allowed each item or all the items to be marked as read, while the unread count has been shown on the bell in the header. The notifications have been created by the reminder engine for the first, second, due-today and overdue reminders, and by the task services for assignments, reassignments, comments and changes of worked hours, so that every important event has been brought to the attention of the people concerned. Each notification has been linked to its task whenever a task had been involved, and a notification that has pointed to a deleted task has been returned without the link, so that a broken page could never have been opened.

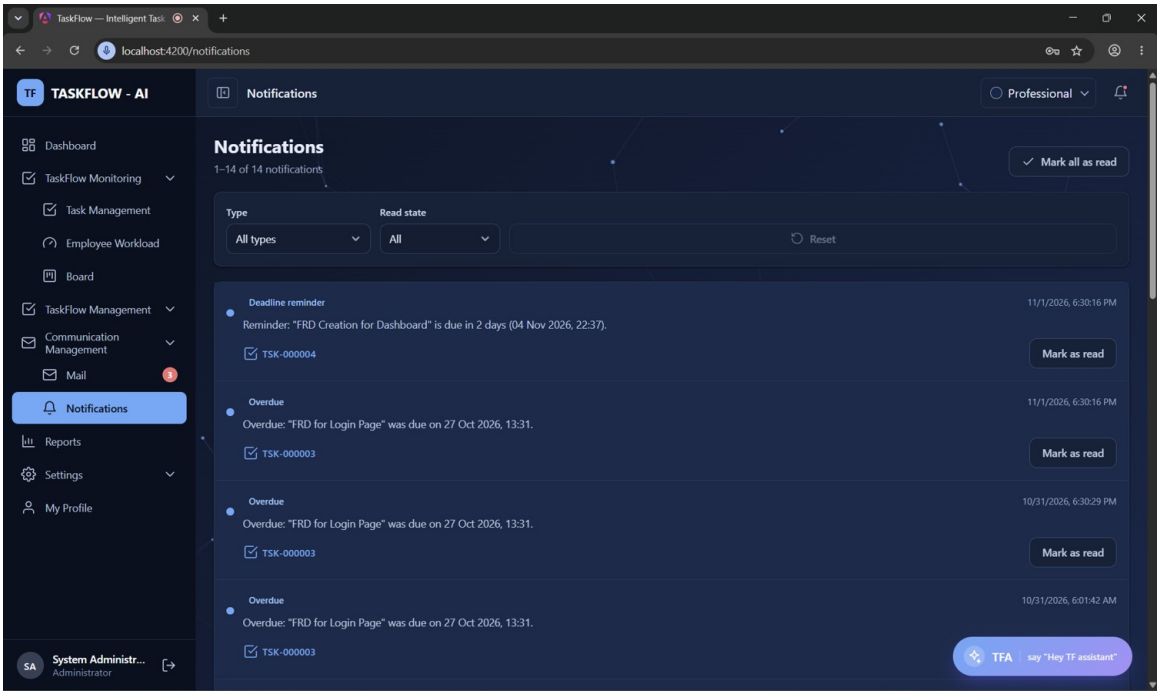


Figure 4.28 Notification Center Screen

The task activity history, which has been shown in Figure 4.29, has displayed every recorded event of a task with the actor, the time, the old value and the new value, and it has been made filterable by the type of event. The events that have been recorded have included the creation and assignment of the task, changes of the deadline and the priority, logged hours, status changes, completion, reassignment, changes of the recipients, comments that had been added, edited or deleted, and the deletion of the task. The history has been kept append-only, so that it could not have been altered by any user, and the only exception has been the correction of a worked-hours entry by an administrator, which has itself been logged as an administrative event.

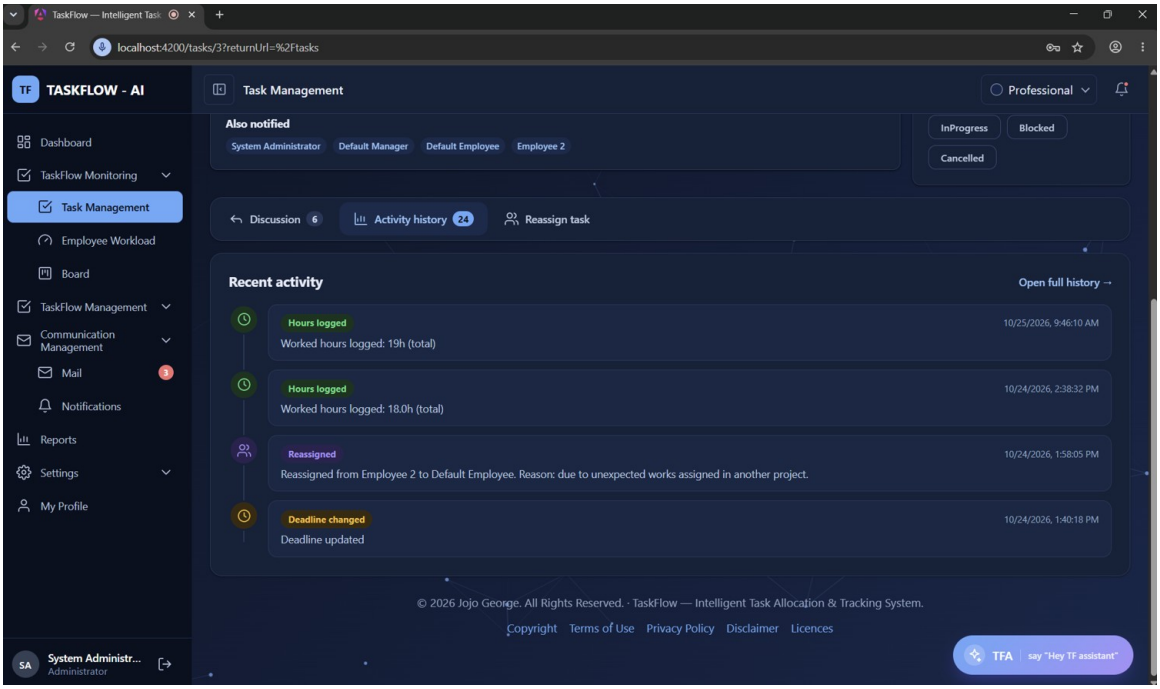


Figure 4.29 Task Activity History Screen

The reports screen, which has been shown in Figure 4.30, has presented the total, pending, completed and overdue counts for the selected deadline range, employee, project, status and priority, and it has been protected by the reports.view permission. The filtered tasks could be downloaded as a CSV file by users who had also been granted the reports.export permission, and the file has been written in UTF-8 according to RFC 4180 with a limit of fifty thousand rows. Every value that had started with a character such as the equals sign has been escaped before it was written to the file, so that a malicious title could never have been executed as a formula when the file was opened in a spreadsheet program.

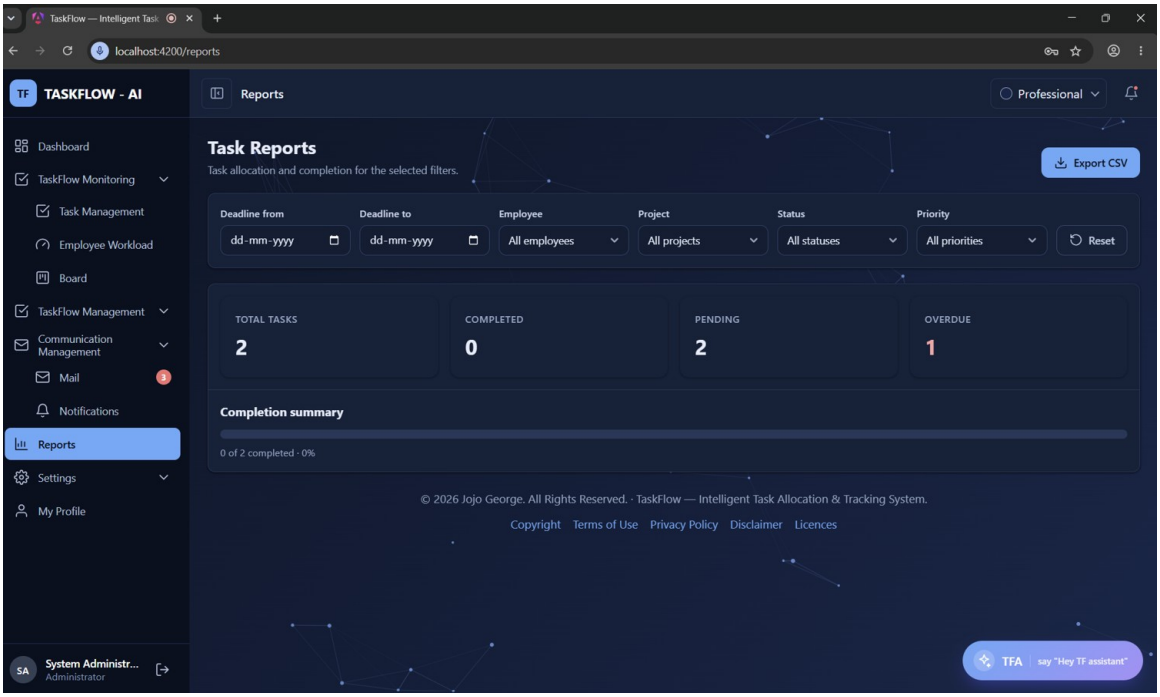


Figure 4.30 Reports Screen With CSV Export

The internal mail inbox, which has been used daily by every user for communication inside the organisation, has been shown in Figure 4.31. The screen has been divided into a folder rail with the Inbox, Sent, Drafts, Starred, Important and Trash views, a paged list of the mails of the selected view with a search box, and a reading pane in which the selected mail has been displayed together with the other mails of the same conversation. Replies, reply-to-all and forwards have been prepared from the reading pane, the unread count of the Inbox has been shown as a badge in the side menu, and the reminder copies that had been sent by the system have been shown with TaskFlow as the sender and with a link to the task.

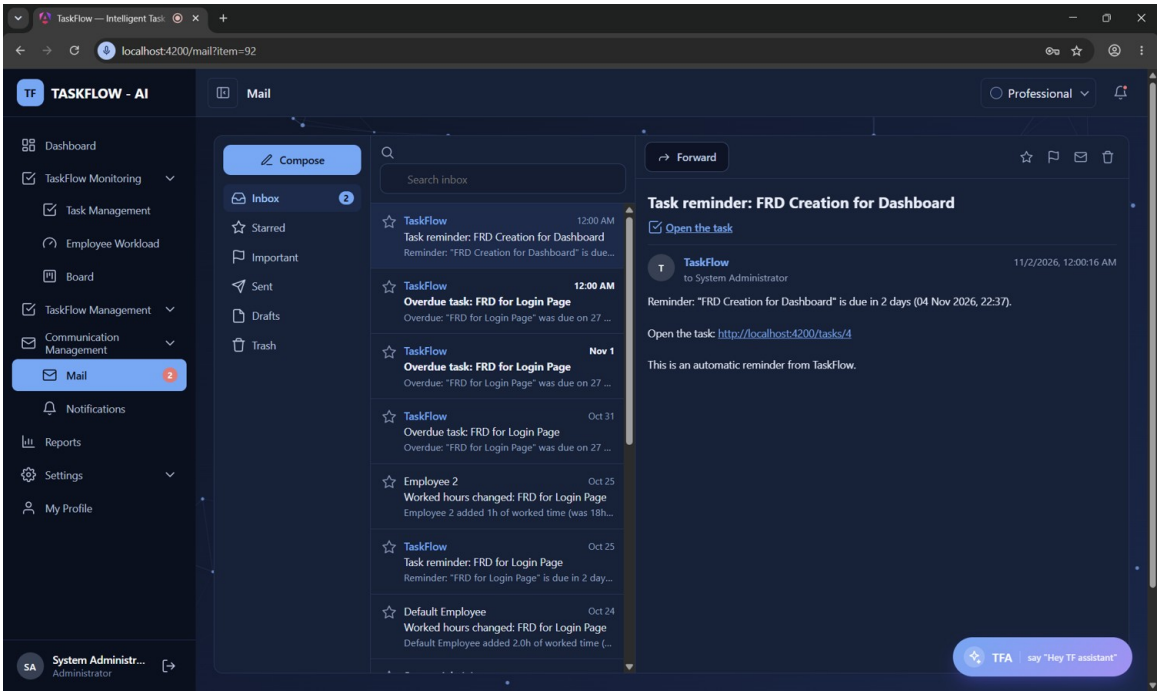


Figure 4.31 Internal Mail Inbox Screen

The TFA assistant panel, which has been made available on every signed-in screen to users who had been granted the assistant.chat permission, has been shown in Figure 4.32. The answers of the assistant have been displayed together with the names of the project documents from which they had been taken, and the ranked lists of tasks that had been produced by the prioritising tool have been shown with links to the corresponding task pages. Whenever an action such as sending a mail or updating the profile had been proposed, an action card with the exact details has been displayed, and the action has been executed only after the confirm button had been pressed by the user, while a microphone button has been provided for voice input in supporting browsers.

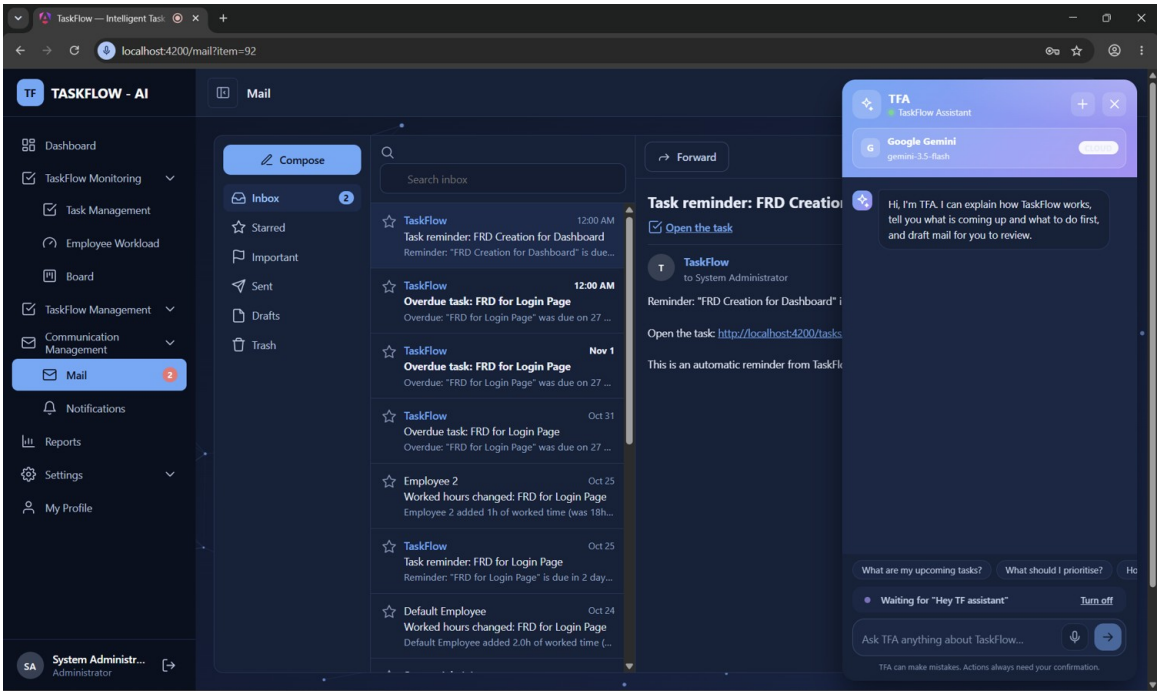


Figure 4.32 TFA Assistant Panel

Apart from the screens, messages have been produced by the system that have been delivered automatically to the users, and the formats of these messages have been designed to be short, specific and consistent. The reminder messages have contained the title of the task and the deadline in the application time zone, the mail copies of the reminders have carried a subject prefix that has indicated the type of the reminder together with a link to the task, and the event notifications have stated what had happened and to which task. The formats of the automatically generated messages have been listed in Table 4.10, and the CSV export has contained one row for each task that had matched the filters selected on the reports screen, encoded in UTF-8 so that it could be opened directly in a spreadsheet program.

Table 4.10 Formats Of Automatic Messages

Event	Notification Text	Mail Subject
Task assigned	A task was assigned to you: {title}	Not applicable
First / second reminder	Reminder: "{title}" is due in {n} day(s) ({date})	Task reminder: {title}
Due today	Reminder: "{title}" is due today at {time}	Task due today: {title}
Overdue	Overdue: "{title}" was due on {date}	Overdue task: {title}
Task reassigned	Reassigned notification to the new and old assignee	Task reassigned: {title}
Comment added / edited	TaskComment notification to the task members	Comment {added / edited}: {title}
Hours changed	System notification to the assignee and recipients	Worked hours changed: {title}

4.1.7. Database Design

Database design has been defined as the process through which the data of the system was organised into tables, columns, keys, relationships and constraints, so that the data has been stored without redundancy, retrieved efficiently and kept consistent at all times. The database of TaskFlow has been designed with the code-first approach of Entity Framework Core and normalised to the third normal form, and it has been made up of thirty-three tables in a single SQLite file. All the timestamps have been stored in UTC and converted to the application time zone only for display, records that have carried history such as users, projects, departments, tasks and comments have been soft deleted through an IsDeleted flag so that the history has remained intact, and the enumerations have been stored either as integers or as text depending on the table. The first entity relationship diagram, which has been drawn for the identity, access and organisation tables, has been shown in Figure 4.33.

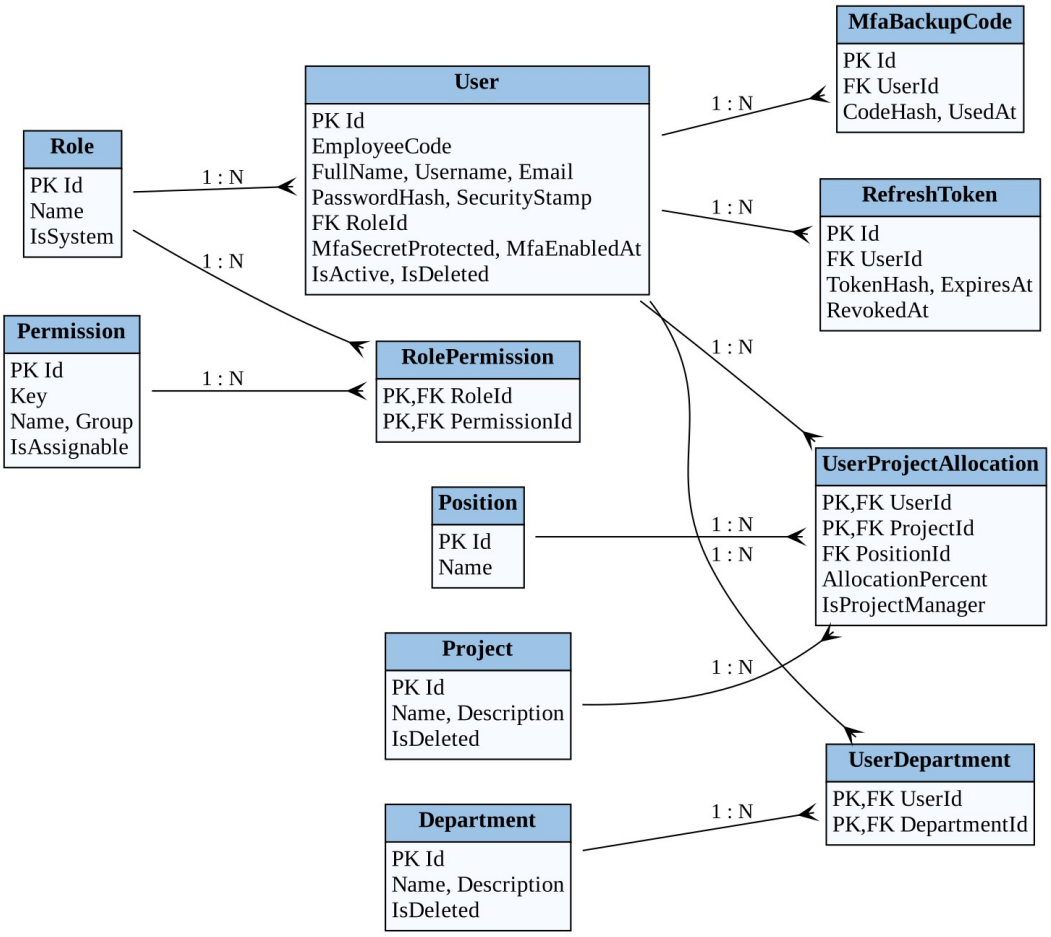


Figure 4.33 Entity Relationship Diagram For Identity And Organisation

The second entity relationship diagram, which has been drawn for the task, reminder, notification and mail tables, has been shown in Figure 4.34. In this diagram the Tasks table has been placed at the centre, because it has been related to the users who had been assigned to it, to the project to which it had belonged, and to its recipients, comments, activity entries,

notifications and reminder history rows. Each reminder history row has been related to at most one system mail through a filtered unique index, and each mail message has been related to its To and Cc recipients, to the personal mailbox items of the sender and the recipients, and to its attachments, so that the complete flow from a task to a delivered reminder could be followed through the relationships.

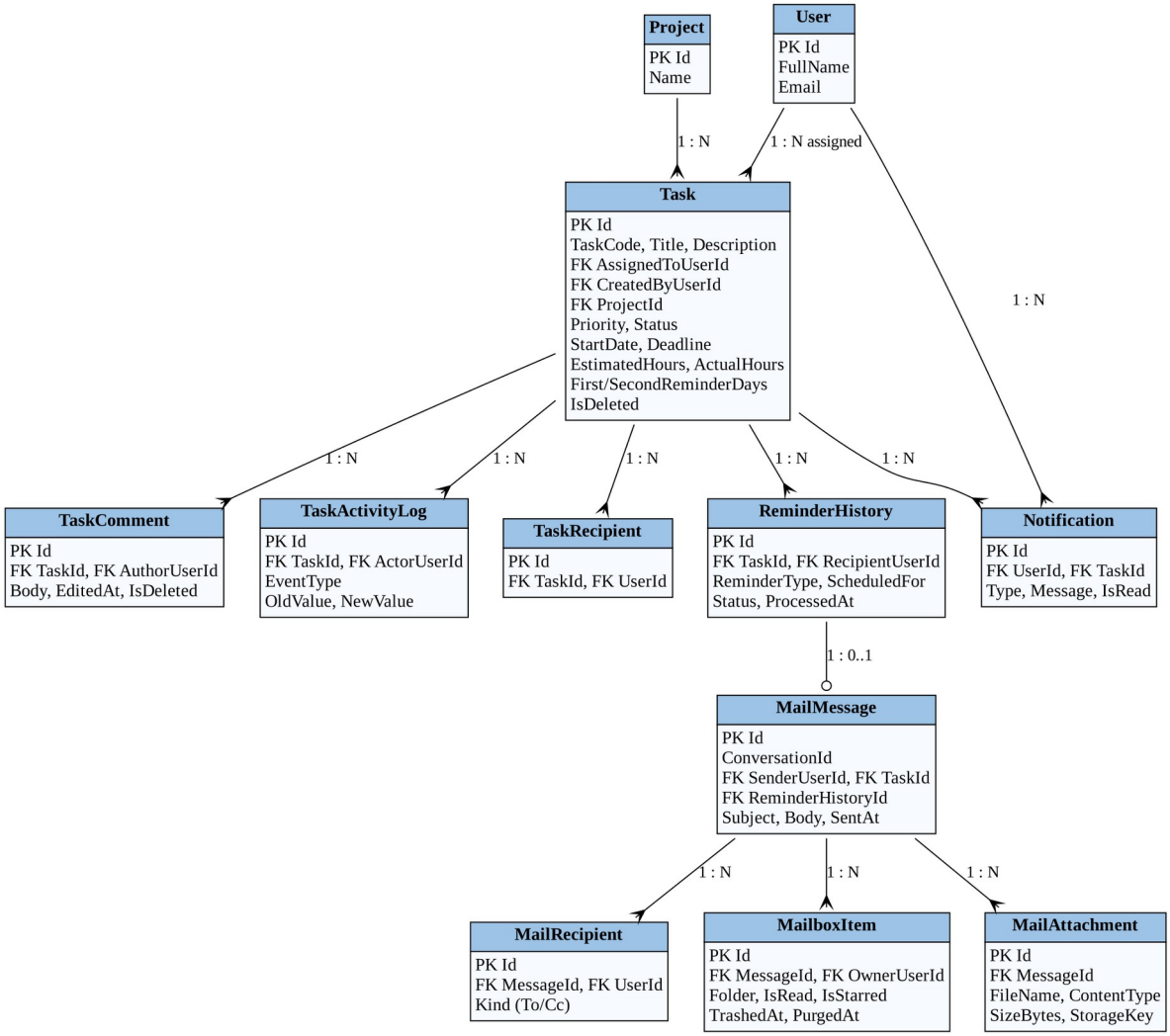


Figure 4.34 Entity Relationship Diagram For Tasks, Reminders And Mail

The integrity of the data has been protected by a number of database level rules in addition to the checks that have been made by the services. Unique indexes have been defined on the user name and email of users, on role, permission, position, project and department names, on the task code, on the pair of task and recipient, and on the combination of task, recipient, reminder type and scheduled day in the reminder history. A check constraint has been used to keep the allocation percentage between one and one hundred, foreign keys have used the Restrict rule for references that have carried history and the Cascade rule for owned child rows, and triggers have been written to reject any update or deletion of the three system roles, their permissions and the permission catalogue. The thirty-three tables have been

grouped by module, and the first eighteen tables, which have belonged to the identity, organisation and task modules, have been listed in Table 4.11.

Table 4.11 List Of Tables For Identity, Organisation And Tasks

No	Table Name	Purpose
1	Users	Employee accounts, credentials, MFA state and status
2	RefreshTokens	Hashed refresh tokens for session renewal
3	PasswordResetGrants	Single-use password reset grants (10 minutes)
4	MfaBackupCodes	Hashed one-time backup codes
5	Roles	System and custom roles
6	Permissions	Read-only catalogue of permission keys
7	RolePermissions	Permissions granted to each role
8	Projects	Projects to which tasks and people belong
9	ManagerProjects	Managers recorded for each project
10	Departments	Organisational departments
11	DepartmentManagers	Managers of each department
12	Positions	Positions such as Developer or QA
13	UserProjectAllocations	Project allocation of users with percentage
14	UserDepartments	Department membership of users
15	Tasks	Tasks with assignee, dates, hours and status
16	TaskRecipients	Users who receive reminders for a task
17	TaskComments	Discussion comments on tasks
18	TaskActivityLogs	Append-only history of task events

The remaining fifteen tables of the database, which have belonged to the communication, settings and assistant modules, have been listed in Table 4.12. In these tables have been held the notifications and the reminder history, the internal mail messages with their recipients, mailbox items and attachments, the runtime settings, the configuration of the AI providers and their models, the conversations, messages, pending actions and audit records of the TFA assistant, and the indexed project documents with their sections and embedding vectors. None of the assistant or document tables has stored any password or API key in plain text, and the document tables have been designed so that they could be rebuilt at any time from the documents folder without any loss of business data.

Table 4.12 List Of Tables For Communication, Settings And Assistant

No	Table Name	Purpose
19	Notifications	In-app notifications of each user
20	ReminderHistory	Delivered reminders with duplicate prevention
21	MailMessages	Internal mail messages and drafts
22	MailRecipients	To and Cc recipients of a message
23	MailboxItems	Personal copy of a mail in a folder
24	MailAttachments	Metadata of uploaded attachments
25	ApplicationSettings	Runtime key and value settings
26	AiProviderConfigs	Configuration of each AI provider
27	AiModels	Built-in and added AI model names
28	AssistantConversations	Conversations of users with TFA
29	AssistantMessages	Messages within a conversation
30	AssistantPendingActions	Actions waiting for confirmation
31	AssistantAudits	Audit trail of assistant actions
32	DocumentSources	Indexed Markdown documents
33	DocumentChunks	Document sections with embedding vectors

4.2 TABLE STRUCTURE

The detailed structure of the principal tables of the database has been presented in this section, and the name, the SQLite storage type, the maximum size, the constraint and a short description of every column have been given. In SQLite, integer, Boolean and enumeration values that have been stored as ordinals have used the INTEGER type, strings, dates, decimals and GUIDs have used the TEXT type, and binary data such as embedding vectors has used the BLOB type, while the sizes that have been shown are the maximum lengths that have been enforced by Entity Framework Core. The Users table, whose structure has been given in Table 4.13, has been used to store the account of every employee together with the password hash, the encrypted authenticator secret, the security stamp, the role and the active and deleted flags, and a global query filter has been applied so that deleted users have been hidden from every normal query.

Table 4.13 Structure Of Users Table

Field Name	Data Type	Size	Constraint	Description
Id	INTEGER	-	Primary Key	Unique identifier of the user
EmployeeCode	TEXT	32	Null	Organisation employee code
FullName	TEXT	200	Not Null	Full name of the employee
Username	TEXT	100	Not Null, Unique	Sign-in user name
Email	TEXT	256	Not Null, Unique	Email on the taskflow.com domain
MobileNumber	TEXT	20	Null	Mobile number in E.164 format
MfaSecretProtected	TEXT	512	Null	Encrypted authenticator secret
MfaEnabledAt	TEXT	-	Null	Time when MFA was enabled
MfaLastUsedStep	INTEGER	-	Concurrency Token	Last accepted TOTP step
MfaFailedAttempts	INTEGER	-	Concurrency Token	Consecutive wrong codes
MfaLockedUntil	TEXT	-	Null	End of MFA lock period
PasswordHash	TEXT	-	Not Null	PBKDF2 password hash
SecurityStamp	TEXT	64	Not Null	Changes to end old sessions
RoleId	INTEGER	-	Foreign Key (Roles)	Role of the user
IsActive	INTEGER	-	Default 1	Inactive users cannot sign in
CreatedAt / UpdatedAt	TEXT	-	Not Null	UTC creation and update time
LastLoginAt	TEXT	-	Null	UTC time of last sign-in
IsDeleted	INTEGER	-	Default 0	Soft delete flag

The Roles table, whose structure has been given in Table 4.14, has been used to store the three system roles and the custom roles that had been created by an administrator through the Role Configuration screen. The system roles have been given the fixed identifiers one for Administrator, two for Manager and three for Employee, and they have been marked by the IsSystem flag, which has been used by SQLite triggers to reject any update or deletion of these rows. The name of each role has been kept unique without regard to the case of the letters through the NOCASE collation, so that two roles named Team Lead and team lead could never have been created, and every user has been linked to exactly one role through a foreign key.

Table 4.14 Structure Of Roles Table

Field Name	Data Type	Size	Constraint	Description
Id	INTEGER	-	Primary Key	1 Administrator, 2 Manager, 3 Employee
Name	TEXT	100	Not Null, Unique (NOCASE)	Name of the role
IsSystem	INTEGER	-	Not Null	True for read-only system roles
CreatedAt	TEXT	-	Not Null	UTC creation time

The Permissions table, whose structure has been given in Table 4.15, has been used to store the fixed catalogue of permission keys, such as tasks.create, reports.export or settings.reminders.manage, together with a display name, a description and the module group in which each key has been shown on the Role Configuration screen. The rows of this table have been inserted at start-up by the access control seeder from the permission catalogue that has been defined in the code, and the whole table has been made read-only through triggers that have rejected every update and deletion. The few keys that could not have been granted from the screen, such as the permission that would have allowed the Administrator role to be given to another user, have been marked by the IsAssignable column.

Table 4.15 Structure Of Permissions Table

Field Name	Data Type	Size	Constraint	Description
Id	INTEGER	-	Primary Key	Unique identifier
Key	TEXT	100	Not Null, Unique	Permission key such as tasks.create
Name	TEXT	200	Not Null	Display name
Description	TEXT	500	Not Null	Purpose of the permission
Group	TEXT	100	Not Null	Module group on the role screen
IsAssignable	INTEGER	-	Default 1	Whether it can be granted

The RolePermissions table, whose structure has been given in Table 4.16, has been used as the link table between roles and permissions, and each of its rows has recorded that one permission had been granted to one role. The pair of the role identifier and the permission identifier has been used as the composite primary key, so that the same permission could never have been granted twice to the same role, and the Cascade rule has been applied to the role while the Restrict rule has been applied to the permission. Triggers have been written to reject the update or deletion of the rows that had belonged to a system role, while new rows have still been allowed, so that newly added keys could be granted automatically to the system roles.

Table 4.16 Structure Of RolePermissions Table

Field Name	Data Type	Size	Constraint	Description
RoleId	INTEGER	-	Primary Key, Foreign Key (Roles)	Role that holds the permission
PermissionId	INTEGER	-	Primary Key, Foreign Key (Permissions)	Permission that is granted

The organisational structure of the company has been stored in the Projects, Departments, Positions, UserProjectAllocations and UserDepartments tables, and the structure of the first of these, the Projects table, has been given in Table 4.17. Every project has been identified by an integer primary key and has been described by a name of up to two hundred characters and an optional description of up to five hundred characters, together with the time of its creation in UTC. Projects have been soft deleted through the IsDeleted flag, and their names have been kept unique only among the live rows through a filtered unique index, so that a deleted project has kept its name for history while a new project with the same name could still be created. A task has been allowed to refer to one project, and a project that had been referenced by tasks has never been removed physically from the database.

Table 4.17 Structure Of Projects Table

Field Name	Data Type	Size	Constraint	Description
Id	INTEGER	-	Primary Key	Unique identifier of the project
Name	TEXT	200	Not Null, Unique (live rows)	Name of the project
Description	TEXT	500	Null	Description of the project
CreatedAt	TEXT	-	Not Null	UTC creation time
IsDeleted	INTEGER	-	Default 0	Soft delete flag

The Departments table, whose structure has been given in Table 4.18, has been used to store the organisational departments to which employees have belonged, and its columns have been designed in the same way as those of the Projects table. Every department has been identified by an integer primary key, has been described by a name of up to two hundred characters and an optional description of up to five hundred characters, and has been soft deleted through the IsDeleted flag. A filtered unique index has kept the names of the live departments unique, and the managers of each department have been recorded in a separate DepartmentManagers link table, so that a department could have been managed by more than one manager when the organisation had required it.

Table 4.18 Structure Of Departments Table

Field Name	Data Type	Size	Constraint	Description
Id	INTEGER	-	Primary Key	Unique identifier of the department
Name	TEXT	200	Not Null, Unique (live rows)	Name of the department
Description	TEXT	500	Null	Description of the department
CreatedAt	TEXT	-	Not Null	UTC creation time
IsDeleted	INTEGER	-	Default 0	Soft delete flag

The Positions table, whose structure has been given in Table 4.19, has been used to store the positions that a user has been able to hold within a project, and six positions, namely Member, Manager, QA, Developer, DevOps and System Admin, have been inserted as seed data by the initial migration. The name of each position has been kept unique without regard to the case of the letters through the NOCASE collation, so that duplicate positions with different capitalisation could never have been created by mistake. A position has been treated only as a descriptive label for the allocation of a user to a project, and no permission has ever been granted through a position, because all access rights have been controlled exclusively through the roles.

Table 4.19 Structure Of Positions Table

Field Name	Data Type	Size	Constraint	Description
Id	INTEGER	-	Primary Key	Unique identifier
Name	TEXT	100	Not Null, Unique (NOCASE)	Member, Manager, QA, Developer, DevOps, System Admin

The UserProjectAllocations table, whose structure has been given in Table 4.20, has recorded the allocation of each user to each project, and the pair of the user identifier and the project identifier has been used as the composite primary key of the table. The percentage of the time of the user that had been allocated to the project has been stored in the AllocationPercent column, and a check constraint has kept this value between one and one hundred, while the service layer has ensured that the total allocation of a user across all projects has not exceeded one hundred percent. The position that the user has held in the project and a flag that has marked the managers of the project have also been stored in this table.

Table 4.20 Structure Of UserProjectAllocations Table

Field Name	Data Type	Size	Constraint	Description
UserId	INTEGER	-	Primary Key, Foreign Key (Users)	Allocated user
ProjectId	INTEGER	-	Primary Key, Foreign Key (Projects)	Project of allocation
AllocationPercent	INTEGER	-	Check 1 to 100	Percentage of time allocated
IsProjectManager	INTEGER	-	Not Null	Marks the project manager
PositionId	INTEGER	-	Foreign Key (Positions)	Position held in the project

The UserDepartments table, whose structure has been given in Table 4.21, has been used to record the membership of users in departments, and the pair of the user identifier and the department identifier has been used as its composite primary key. A user has been allowed to belong to several departments at the same time, and the Cascade rule has been applied to both foreign keys, so that the membership rows have been removed together with the user or the department if either of them had ever been deleted physically. A global query filter has hidden every membership row whose user or department had been soft deleted, so that deleted departments and former employees have never appeared in the member lists of the management screens.

Table 4.21 Structure Of UserDepartments Table

Field Name	Data Type	Size	Constraint	Description
UserId	INTEGER	-	Primary Key, Foreign Key (Users)	Member user
DepartmentId	INTEGER	-	Primary Key, Foreign Key (Departments)	Department of membership

The Tasks table, whose structure has been given in Table 4.22, has been designed as the central table of the system, and every task has been stored in it together with its assignee, creator, project, priority, dates, estimated and actual hours, status and reminder intervals. The task code has been generated from the identifier in the form TSK followed by six digits and has been protected by a unique index, while indexes have also been created on the deadline and on the assignee so that the reminder engine and the task lists could find the required rows quickly. The ActualHours column has been declared as a concurrency token, so that two users logging hours at the same moment could never have overwritten each other, and the overdue state and the display stage of a task have never been stored, because they have been computed whenever the task has been read.

Table 4.22 Structure Of Tasks Table

Field Name	Data Type	Size	Constraint	Description
Id	INTEGER	-	Primary Key	Unique identifier of the task
TaskCode	TEXT	32	Not Null, Unique	Code such as TSK-000042
Title	TEXT	200	Not Null	Title of the task
Description	TEXT	4000	Null	Detailed description
AssignedToUserId	INTEGER	-	Foreign Key (Users), Index	Current assignee
CreatedByUserId	INTEGER	-	Foreign Key (Users)	User who created the task
ProjectId	INTEGER	-	Foreign Key (Projects)	Project of the task
Priority	INTEGER	-	Not Null	Low, Medium, High, Critical
StartDate	TEXT	-	Not Null	UTC start date and time
Deadline	TEXT	-	Not Null, Index	UTC deadline
EstimatedHours	TEXT	-	Not Null	Estimated effort in hours
ActualHours	TEXT	-	Concurrency Token	Hours worked so far
Status	INTEGER	-	Not Null	ToDo, InProgress, Blocked, Completed, Cancelled, InQA
FirstReminderDays	INTEGER	-	Default 2	First reminder interval
SecondReminderDays	INTEGER	-	Default 1	Second reminder interval
OverdueReminderFrequency	INTEGER	-	Default Daily	Daily or Weekly
CreatedAt / UpdatedAt	TEXT	-	Not Null	UTC timestamps
CompletedAt / CancelledAt	TEXT	-	Null	Time of terminal status
IsDeleted	INTEGER	-	Default 0	Soft delete flag

The TaskRecipients table, whose structure has been given in Table 4.23, has been used to store the users who have been informed about a task and who have received its reminders, and the assignee of a task has always been included in this table as one of the recipients. A unique index on the pair of the task and the user has ensured that the same person could never have been added twice to the same task, and at most fifty additional recipients have been allowed by the validation rules. When a task had been reassigned, the new assignee has been added as a recipient if necessary, while the previous assignee has remained a recipient, so that both people have continued to be informed about the progress of the task.

Table 4.23 Structure Of TaskRecipients Table

Field Name	Data Type	Size	Constraint	Description
Id	INTEGER	-	Primary Key	Unique identifier
TaskId	INTEGER	-	Foreign Key (Tasks), Unique with UserId	Task concerned
UserId	INTEGER	-	Foreign Key (Users)	Recipient of reminders

The TaskComments table, whose structure has been given in Table 4.24, has been used to store the discussion on each task, and every comment has been linked to its task and to its author through foreign keys. The body of a comment has been limited to four thousand characters, the time of creation has been stored in UTC, and an index on the task and the creation time has been created so that the comments of a task could be listed quickly in the order in which they had been written. A comment has been editable and deletable only by its author, an edit has set the EditedAt column, a deletion has only set the IsDeleted flag, and the earlier text has been preserved in the activity history.

Table 4.24 Structure Of TaskComments Table

Field Name	Data Type	Size	Constraint	Description
Id	INTEGER	-	Primary Key	Unique identifier
TaskId	INTEGER	-	Foreign Key (Tasks), Index	Task concerned
AuthorUserId	INTEGER	-	Foreign Key (Users)	Author of the comment
Body	TEXT	4000	Not Null	Text of the comment
CreatedAt	TEXT	-	Not Null	UTC creation time
EditedAt	TEXT	-	Null	Time of last edit by author
IsDeleted	INTEGER	-	Default 0	Soft delete flag

The TaskActivityLogs table, whose structure has been given in Table 4.25, has been used to store the complete history of events of every task, and each entry has recorded the type of the event, the user who had caused it, the old value, the new value, a readable message and the time in UTC. The event type has been stored as text so that entries such as Created, Assigned, StatusChanged, DeadlineChanged, Reassigned, HoursLogged and CommentEdited have remained readable directly in the database. The table has been kept append-only, and the only correction that has been allowed is the correction of a worked-hours entry by an administrator, which has itself been recorded as a separate administrative event in the same table.

Table 4.25 Structure Of TaskActivityLogs Table

Field Name	Data Type	Size	Constraint	Description
Id	INTEGER	-	Primary Key	Unique identifier
TaskId	INTEGER	-	Foreign Key (Tasks), Index	Task concerned
EventType	TEXT	32	Not Null	Created, StatusChanged, Reassigned, etc.
ActorUserId	INTEGER	-	Foreign Key (Users)	User who caused the event
OldValue	TEXT	-	Null	Value before the change
NewValue	TEXT	-	Null	Value after the change
Message	TEXT	-	Not Null	Readable description of the event
CreatedAt	TEXT	-	Not Null	UTC time of the event

The communication features of the system have been supported by the Notifications, ReminderHistory, MailMessages, MailRecipients, MailboxItems and MailAttachments tables, and the structure of the Notifications table has been given in Table 4.26. Each notification has been stored for exactly one user, has optionally been linked to a task, and has carried a type such as TaskAssigned, Reassigned, DeadlineReminder, Overdue, System or TaskComment, which has been stored as text so that its meaning has remained clear. A composite index on the user and the read flag has been created, because the unread count that has been shown on the bell of every page has been requested very frequently by the browser. The notifications of a user have been deleted automatically through the Cascade rule only if the user had been removed physically, which has never happened in practice because users have been soft deleted.

Table 4.26 Structure Of Notifications Table

Field Name	Data Type	Size	Constraint	Description
Id	INTEGER	-	Primary Key	Unique identifier
UserId	INTEGER	-	Foreign Key (Users), Index	User who receives it
TaskId	INTEGER	-	Foreign Key (Tasks), Null	Related task
Type	TEXT	32	Not Null	TaskAssigned, DeadlineReminder, Overdue, etc.
Message	TEXT	-	Not Null	Text of the notification
IsRead	INTEGER	-	Not Null	Read flag
CreatedAt	TEXT	-	Not Null	UTC creation time

The ReminderHistory table, whose structure has been given in Table 4.27, has been used to record every reminder that has been delivered by the automatic reminder engine, and each row has identified the task, the recipient, the reminder type and the scheduled day. The scheduled day has been stored as the UTC instant of local midnight of the reminder day in the

application time zone, and the unique index on the task, recipient, reminder type and scheduled day columns has been used as the mechanism that has prevented duplicate reminders. Each row has been inserted with an ON CONFLICT DO NOTHING clause, so that repeated passes of the scheduler and restarts of the server could never have produced a second copy of the same reminder.

Table 4.27 Structure Of ReminderHistory Table

Field Name	Data Type	Size	Constraint	Description
Id	INTEGER	-	Primary Key	Unique identifier
TaskId	INTEGER	-	Foreign Key (Tasks)	Task reminded
RecipientUserId	INTEGER	-	Foreign Key (Users)	User reminded
ReminderType	TEXT	32	Not Null	First, Second, DueToday, Overdue
ScheduledFor	TEXT	-	Not Null	UTC instant of local midnight of the day
Status	TEXT	32	Not Null	Delivery status (Sent)
ProcessedAt	TEXT	-	Null	UTC time of delivery
DeliveryAttempts	INTEGER	-	Not Null	Number of attempts
(TaskId, RecipientUserId, ReminderType, ScheduledFor)	-	-	Unique Index	Duplicate prevention

The MailMessages table, whose structure has been given in Table 4.28, has been used to store every internal mail and every draft only once, together with its subject, its plain text body of up to twenty thousand characters and the times of creation, update and sending. A conversation identifier has been shared by a mail and all its replies and forwards, the parent message has been recorded for replies, and the sender has been left empty for the system mails that had been generated by the reminder engine. The filtered unique index on the reminder history column has ensured that at most one mail has been created for each reminder, and a mail without a sending time has been treated as a draft of its author.

Table 4.28 Structure Of MailMessages Table

Field Name	Data Type	Size	Constraint	Description
Id	INTEGER	-	Primary Key	Unique identifier
ConversationId	TEXT	-	Not Null, Index	GUID shared by a thread
ParentMessageId	INTEGER	-	Foreign Key (self), Null	Message replied to
SenderUserId	INTEGER	-	Foreign Key (Users), Null	Sender; null for system mail
IsSystem	INTEGER	-	Not Null	True for reminder mails
Subject	TEXT	200	Not Null	Subject line
Body	TEXT	20000	Not Null	Plain text body
TaskId	INTEGER	-	Foreign Key (Tasks), Null	Related task
ReminderHistoryId	INTEGER	-	Foreign Key, Unique when set	Related reminder
DraftTo / DraftCc	TEXT	4000	Null	Addresses of a draft
CreatedAt / UpdatedAt	TEXT	-	Not Null	UTC timestamps
SentAt	TEXT	-	Null	Null for a draft

The MailboxItems table, whose structure has been given in Table 4.29, has been used to store the personal copy of a mail for each sender and recipient, so that every user has been able to organise the mails independently of the other users. Each item has recorded the owner, the folder, which has been one of Inbox, Sent or Drafts, and the personal read, starred and important flags, while the Trash view has been produced from the TrashedAt column and the deletion forever has been recorded in the PurgedAt column. A composite index on the owner, folder, purge time, trash time and sort time has been created, so that the mails of each view could be listed quickly, and the deletion of a mail by one user has not affected the copies of the other users.

Table 4.29 Structure Of MailboxItems Table

Field Name	Data Type	Size	Constraint	Description
Id	INTEGER	-	Primary Key	Unique identifier
MessageId	INTEGER	-	Foreign Key (MailMessages)	Message concerned
OwnerUserId	INTEGER	-	Foreign Key (Users), Index	Owner of this copy
Folder	TEXT	16	Not Null	Inbox, Sent or Drafts
IsRead / IsStarred / IsImportant	INTEGER	-	Not Null	Personal flags
TrashedAt	TEXT	-	Null	Time moved to Trash
PurgedAt	TEXT	-	Null	Time deleted forever
SortAt	TEXT	-	Not Null	Sort time of the item

The settings and the assistant features of the system have been supported by the ApplicationSettings, AiProviderConfigs, AiModels and assistant tables, and the structure of the ApplicationSettings table has been given in Table 4.30. Only non-secret values have been stored in the settings table as key and value pairs, and each key, such as Application.Timezone, Reminder.Enabled, Reminder.DefaultFirstDays or Notification.MailEnabled, has been protected by a unique index and grouped into the Application, Reminder or Notification category. The rows of this table have been inserted by a settings initialiser at start-up whenever they had been missing, and existing values have never been overwritten, so that the changes that had been made by the administrator have always been preserved across restarts and upgrades of the application.

Table 4.30 Structure Of ApplicationSettings Table

Field Name	Data Type	Size	Constraint	Description
Id	INTEGER	-	Primary Key	Unique identifier
Key	TEXT	128	Not Null, Unique	Setting key such as Reminder.Enabled
Value	TEXT	1024	Not Null	Invariant culture value
Category	TEXT	64	Not Null	Application, Reminder or Notification
UpdatedByUserId	INTEGER	-	Null	User who last changed it
UpdatedAt	TEXT	-	Not Null	UTC time of last change

The AiProviderConfigs table, whose structure has been given in Table 4.31, has been used to store the configuration of each AI provider, namely Google Gemini, OpenAI and Ollama, with exactly one row for each provider through a unique index on the provider kind. The model that had been selected for the provider, the address that has been used by a local Ollama server and the enabled flag have been stored in plain columns, while the API key has been encrypted with ASP.NET Core Data Protection before it was stored. Only the last four characters of the key have ever been shown on the TFA settings screen, and the decrypted key has been held in memory only for the duration of a single call to the provider.

Table 4.31 Structure Of AiProviderConfigs Table

Field Name	Data Type	Size	Constraint	Description
Id	INTEGER	-	Primary Key	Unique identifier
Kind	TEXT	16	Not Null, Unique	Gemini, OpenAI or Ollama
IsEnabled	INTEGER	-	Not Null	Whether the provider is in use
ModelName	TEXT	100	Not Null	Selected model
BaseUrl	TEXT	300	Null	Address used by Ollama
EncryptedApiKey	TEXT	-	Null	API key encrypted by Data Protection
KeyLast4	TEXT	4	Null	Last four characters for display
UpdatedByUserId	INTEGER	-	Null	User who last changed it
UpdatedAt	TEXT	-	Not Null	UTC time of last change

The AssistantPendingActions table, whose structure has been given in Table 4.32, has been used to hold the actions that had been prepared by the TFA assistant but that had not yet been confirmed by the user, such as sending a mail, updating the own profile or changing the role of a user. Each action has been identified by a GUID, which has been used in the confirm address, and the exact details that had been shown to the user have been stored as JSON, so that precisely those details have been executed on confirmation. The status of each action has moved from Pending to Confirmed, Cancelled, Expired, Refused or Failed, and an action that had not been confirmed before its expiry time has never been executed.

Table 4.32 Structure Of AssistantPendingActions Table

Field Name	Data Type	Size	Constraint	Description
Id	TEXT	-	Primary Key (GUID)	Identifier used in confirm URL
UserId	INTEGER	-	Foreign Key (Users), Index	User for whom it was prepared
ConversationId	INTEGER	-	Not Null	Conversation of origin
Kind	TEXT	32	Not Null	SendMail, UpdateOwnProfile, ChangeUserRole
PayloadJson	TEXT	-	Not Null	Exact details shown to the user
Status	TEXT	16	Not Null	Pending, Confirmed, Cancelled, Expired
CreatedAt / ExpiresAt	TEXT	-	Not Null	UTC creation and expiry time
ResolvedAt	TEXT	-	Null	UTC time of resolution

The DocumentChunks table, whose structure has been given in Table 4.33, has been used to store the sections of the project documents that have been searched by the TFA

assistant, and each chunk has been linked to its source document in the DocumentSources table. The heading path above each chunk, its position within the document and the section text that has been given to the model have been stored together with the embedding vector, which has been held as a normalised array of float32 values with seven hundred and sixty-eight dimensions in a BLOB column. A full text index has also been maintained over the chunks through SQLite triggers, so that exact identifiers could be found by keyword search alongside the vector similarity search.

Table 4.33 Structure Of DocumentChunks Table

Field Name	Data Type	Size	Constraint	Description
Id	INTEGER	-	Primary Key	Unique identifier
DocumentSourceId	INTEGER	-	Foreign Key (DocumentSources)	Source document
Ordinal	INTEGER	-	Not Null	Position within the document
HeadingPath	TEXT	1000	Not Null	Headings above the chunk
Text	TEXT	-	Not Null	Section text given to the model
Embedding	BLOB	-	Not Null	Normalised float32 vector
Dimensions	INTEGER	-	Not Null	Vector length (768)

CHAPTER 5
SYSTEM TESTING AND IMPLEMENTATION

System testing has been understood as the process in which the system has been executed with the intention of finding errors and of confirming that the requirements which had been specified have been met, and system implementation has been understood as the process in which the tested system has been installed in its working environment and put into use. In TaskFlow, testing has not been treated as a separate final step, because it has been carried out continuously alongside development, and most of the tests have been automated so that they could be repeated after every change to the code. The automated test suites have consisted of a unit test project, an integration test project and the front end specification tests, and these suites have been complemented by manual acceptance, output and validation testing, while the testing activities, the security measures, the implementation and the enhancements of the system have been described in this chapter.

5.1 SYSTEM TESTING

The testing strategy of TaskFlow has been designed so that every layer of the system has been verified at the level that has been most appropriate for it. Pure business rules such as the status transition table, the reminder calculation and the interval ordering have been tested directly with unit tests, services have been tested with hand-written fakes for their dependencies, and the HTTP endpoints have been tested end to end through the real request pipeline with the WebApplicationFactory of ASP.NET Core. The front end components and services have been tested with Jasmine and Karma after a script that checks the folder structure had been run, and architecture tests have been used to verify that the layering rules of the back end have never been broken. The overall size of the automated test suites has been shown in Table 5.1, and all the tests have passed in the latest recorded run of the suites.

Table 5.1 Summary Of Automated Test Suites

Test Suite	Tool	Test Files	Tests Executed
TaskFlow.UnitTests	xUnit, NetArchTest	93 classes	1,042 passed
TaskFlow.IntegrationTests	xUnit, WebApplicationFactory	46 classes	467 passed
Frontend specifications	Jasmine, Karma	99 spec files	1,099 passed
Structure check	check-frontend-structure.mjs	1 script	Passed
Total			**2,608 passed**

5.1.1. Unit Testing

Unit testing has been defined as the testing of the smallest testable parts of the program, such as individual methods and classes, which have been isolated from the rest of the system during the test. In TaskFlow, unit tests have been written with the xUnit framework for the pure rule classes, the services, the validators, the seeders, the integrations and the architecture rules, and the tests have been organised in folders that have mirrored the modules of the code. A fake time provider has been used so that the reminder calculations could be tested for any date and time zone, including the days on which daylight saving time has begun or ended, and a scripted chat provider has been used so that the assistant could be tested without any real AI model being called. Some representative unit test cases and the results that have been obtained for them have been listed in Table 5.2 for reference.

Table 5.2 Sample Unit Test Cases

No	Test Case	Expected Result	Result
1	Move a task from Completed to In Progress	Transition refused	Pass
2	Move a task from In Progress to In QA	Transition allowed	Pass
3	Deadline 2 days ahead with first interval 2	FirstReminder returned	Pass
4	Equal first and second intervals	Only one reminder returned	Pass
5	Task completed before the deadline	No reminder returned	Pass
6	Overdue task with weekly frequency on day 3	No reminder returned	Pass
7	TOTP code reused within the same step	Code rejected	Pass
8	Five consecutive wrong MFA codes	Account locked for 5 minutes	Pass
9	Create task with deadline before start date	Validation error returned	Pass
10	Controller referring to a repository directly	Architecture test fails the build	Pass
11	CSV value starting with = sign	Value escaped against formula injection	Pass
12	Chat message containing an API key	Message refused before model call	Pass

5.1.2. Integration Testing

Integration testing has been carried out to verify that the units which had been tested separately have worked correctly when they have been combined, and that the data has been passed correctly across the interfaces between them. In TaskFlow, the integration tests have started the complete API in memory through the WebApplicationFactory with a separate test

database, and real HTTP requests have been sent through the complete middleware pipeline, including authentication, the two-step verification gate, the permission policies, the validation filter and the exception handler. These tests have confirmed that every endpoint has refused callers who have not held its permission key, that users have not been able to read the tasks of others or the analytics of another team, that errors have been returned in the ProblemDetails format, and that the last active administrator could not be demoted, deactivated or deleted, and some representative integration test cases have been listed in Table 5.3.

Table 5.3 Sample Integration Test Cases

No	Test Case	Expected Result	Result
1	POST /auth/login with wrong password	401 with empty body	Pass
2	API call before MFA setup	403 with code mfa_setup_required	Pass
3	Employee calls GET /reports	403 Forbidden	Pass
4	Employee opens task of another user	403 or 404 returned	Pass
5	Manager reads analytics of another team	403 Forbidden	Pass
6	Refresh with a revoked refresh token	401 and cookie cleared	Pass
7	Change of custom role permissions	Holders signed out on next call	Pass
8	Delete the last active administrator	409 Conflict refused	Pass
9	Scheduler pass run twice for the same day	Reminder stored only once	Pass
10	Send mail with attachment to two users	Each recipient has one Inbox item	Pass

5.1.3. User Acceptance Testing

User acceptance testing has been performed as the final level of testing, in which the intended users of the system have verified that it has met their business needs and that it could be accepted for regular use. For TaskFlow, the acceptance criteria that had been recorded in the approved Business Requirements Document have been used as the basis of this testing, and each criterion has been verified by performing the corresponding business scenario on the running application with sample users in the administrator, manager and employee roles. Family members and friends who have not been software developers have also been asked to perform common tasks such as signing in, setting up the authenticator, creating a task, moving it on the board and reading the reminders, and their observations have been used to improve the labels, the messages and the layout of some screens, while a selection of the acceptance test results has been given in Table 5.4.

Table 5.4 Sample User Acceptance Test Results

No	Acceptance Scenario	Performed By	Status
1	Manager creates and assigns a task on one screen	Manager	Accepted
2	Assignee is notified immediately after saving	Employee	Accepted
3	Same status is shown in list, board and dashboard	Employee	Accepted
4	Reminders arrive 2 days, 1 day before and on deadline	All	Accepted
5	Overdue reminder arrives every day until completion	Employee	Accepted
6	Reassignment requires and records a reason	Manager	Accepted
7	Report filters and CSV export give correct totals	Manager	Accepted
8	Sign-in requires password and authenticator code	All	Accepted
9	Employee cannot see other employees tasks	Employee	Accepted
10	TFA prepares a mail and sends only after confirmation	Employee	Accepted

5.1.4. Output Testing

Output testing has been carried out to verify that the outputs which have been produced by the system have been correct, complete and presented in the format that the users have required. In TaskFlow, the figures of the dashboards have been compared with the counts that had been obtained by querying the database directly for the same filters, the totals of the reports screen have been compared with the number of rows in the exported CSV file, and the CSV file has been opened in a spreadsheet program to confirm that special characters, commas and line breaks in the titles have been handled correctly. The texts of the reminder notifications and mails have been checked to ensure that the deadlines have been shown in the application time zone, that the overdue tasks have been highlighted in the same way on the list, the board and the dashboards, and that the paging of the lists has shown the correct number of rows on each page.

A particular effort has been made to test the outputs of the reminder engine, because the reminders have been regarded as the most visible output of the system for the employees who have used it every day. A set of sample tasks has been created with deadlines that were two days away, one day away, on the same day and several days in the past, and the engine has been allowed to run over a number of scheduler ticks and through a restart of the API. The notification centre and the mail inbox of each recipient have then been examined, and it has been confirmed that each reminder has appeared exactly once, that no reminder has been sent for completed or cancelled tasks, that a change of the deadline has been reflected in the next

pass, and that the overdue reminders have followed the daily or weekly frequency that had been set for each task.

5.1.5. Validation Testing

Validation testing has been carried out to verify that the system has accepted valid data and has rejected invalid data with clear and helpful messages, so that the integrity of the stored information has been preserved at all times. In TaskFlow, every input form has been tested with valid values, with empty required fields, with values that have exceeded the maximum lengths, with values outside the permitted ranges and with combinations of values that have broken a business rule. Since validation has been enforced on the server through FluentValidation and the service rules, the same tests have also been performed directly against the API without the browser, in order to confirm that the server has rejected invalid data even when the client side checks had been bypassed, and some of the validation test cases together with the messages that have been returned have been shown in Table 5.5.

Table 5.5 Sample Validation Test Cases

Input	Invalid Value	System Response
Task deadline	Before the start date	Deadline must be after the start date.
Estimated hours	0 or negative	Estimated hours must be a positive number.
Assigned employee	Inactive employee	The assigned employee does not exist or is not active.
Additional recipients	51 users	A task can have at most 50 additional recipients.
Second reminder days	Greater than first	The second reminder cannot come before the first reminder.
Reassignment reason	Empty	A reassignment reason is required.
Worked hours entry	More than 24	Rejected: an entry must be more than 0 and at most 24 hours.
Comment edit	Edited by another user	Only the author can edit this comment.
Status change	Completed to In Progress	Cannot move a task from Completed to InProgress.
New password	Fewer than 6 characters	Rejected: a password must have 6 to 128 characters.

5.2 SYSTEM SECURITY

Security has been treated as a primary requirement of TaskFlow, because the work data, the workload and the personal details of all the employees of an organisation have been held by the system. The first line of protection has been provided by authentication, in which

passwords have been stored only as salted PBKDF2 hashes, two-step verification with an authenticator application has been made mandatory for every user and has been enforced on the server by a middleware gate that the browser cannot bypass, and each time-based code has been allowed to be used only once. Ten single-use backup codes have been issued when two-step verification has been enabled, wrong codes have led to a temporary lock, and the same message has been returned by the sign-in and recovery endpoints for every failure so that an attacker could not discover which accounts have existed. Self-service password recovery has been made possible only with a valid authenticator code, and it has been limited by attempt counters per account and per network address.

Sessions have been protected by short lived JWT access tokens of thirty minutes that have been kept only in the memory of the browser, and by refresh tokens of seven days that have been stored as SHA-256 hashes, rotated on every use and sent only in an HttpOnly, Secure and SameSite=Strict cookie, which has also protected the application against cross-site request forgery. On every request, the role and the permissions of the user have been reloaded by the server and the security stamp in the token has been compared with the one in the database, so that a password change, a role change, a deactivation or a change of the permissions of a custom role has ended the existing sessions at once. Authorisation has been made permission based with one policy for each of the seventy permission keys, a resource check has ensured that only the assignee, the creator, the recipients and authorised users could open a task, and every refused request has been recorded in the log.

The data itself has been protected in several complementary ways, each of which has been chosen to close a particular route through which the data could otherwise have been exposed. All database access has taken place through Entity Framework Core with parameterised queries, so that SQL injection has not been possible, and the wildcard characters of LIKE searches have been escaped. The authenticator secrets of the users and the API keys of the AI providers have been encrypted with ASP.NET Core Data Protection, whose key ring has itself been protected with DPAPI on Windows, secrets such as the JWT signing key have been kept out of the source code, and request bodies, which might have contained passwords or codes, have never been written to the log. Mail attachments have been checked against an extension list and have been downloaded with a server chosen content type, cross-site scripting has been prevented by the Angular templates, redirects have not been followed by the AI provider clients, and any chat message that has looked like a password or a key has been refused by the assistant before it could reach an external service.

5.3 SYSTEM IMPLEMENTATION

System implementation has been understood as the stage in which the tested system has been installed on the target machine, the initial data has been loaded, the users have been trained and the system has been put into operation. For TaskFlow, the implementation has been carried

out by installing the .NET 10 runtime and Node.js on the server, installing the front end packages with the npm install command, storing the JWT signing key and the optional Gemini API key in the user secrets store or in environment variables, and starting the API with the dotnet run command, which has also started the Angular development server through the SPA proxy. On the first start, the data folder has been created by the application, all the database migrations have been applied, the permission catalogue, the system roles, the default settings and the built-in AI models have been inserted, and the reminder scheduler and the document indexer have been started without any manual database work.

For production use, the Angular application has been compiled with the ng build command and its output folder has been served as static files, while the API has been run behind HTTPS on the server or inside a Docker container that has been built from the provided Dockerfile, and the allowed origins and the base address of the application have been set in the environment file. The changeover from the manual system has been planned as a direct changeover, in which the departments, projects and employee accounts have first been created by the administrator, the authenticator has then been set up by each employee on the first visit, and all new tasks have been created in TaskFlow by the managers from a chosen date. Short demonstrations of the dashboards, the task screens, the board and the reminder settings have been considered sufficient as training, because the screens have followed familiar conventions, and regular backups of the data folder have been recommended to the administrator.

5.4 SYSTEM ENHANCEMENT

System enhancement has referred to the improvements that have been made to a system after its initial design in order to add new capabilities, to improve its quality or to adapt it to changing needs of the organisation. During the development of TaskFlow itself, a number of enhancements have been made beyond the original requirement, and each enhancement has been recorded through a change proposal, a design and a task list before it was implemented. The original requirement had suggested e-mail notifications through an SMTP server, which has been replaced by an internal mail module that has worked offline and has needed no stored mail credentials, and the fixed role checks have been replaced by a configurable permission catalogue with custom roles. Mandatory two-step verification, authenticator based password recovery, projects with positions and allocation percentages, the In QA status, comment editing, worked hours logging and the analytics dashboards have also been added to the system during this stage.

The most significant enhancement has been the TFA assistant, which has been added as an AI based help and action panel on every signed-in screen of the application. Google Gemini, OpenAI or a locally installed Ollama model has been made available to the assistant, and the provider has been configured by the administrator through the TFA settings screen, while questions about TaskFlow have been answered only from the project documents through a

hybrid search that has combined vector similarity with SQLite full text search. The open tasks of the user could be listed and prioritised, colleagues could be found, a mail could be drafted and an update of the profile could be proposed, and every such action has been executed only after it had been reviewed and confirmed by the user. Voice input through the Web Speech API of the browser and an audit trail of all the actions have also been provided, and these enhancements have made the system considerably more useful than had been envisaged in the original specification.

CHAPTER 6
CONCLUSION AND FURTHER ENHANCEMENT

This chapter has been written to summarise the work that has been carried out in the project, to evaluate the extent to which the objectives have been achieved, and to identify the directions in which the system could be enhanced in future. The development of TaskFlow has provided an opportunity in which the concepts of software engineering, database design, web technologies, security and software testing that had been studied during the MCA programme have been applied to a realistic problem of an office. A complete documented life cycle, from the business requirements to a tested and deployable product, has also been followed during the project, and the conclusions that have been drawn together with the enhancements that have been planned have been presented in the sections that follow in this chapter.

6.1 CONCLUSION

TaskFlow – Intelligent Task Allocation and Tracking System has been successfully designed, developed, tested and documented as a secure, role-based web application for allocating and tracking the work of employees. All the objectives that had been set for the system have been met, since tasks have been created and assigned on one screen, their progress has been made visible through a fixed status life cycle on the list, the board and the dashboards, and automatic reminders have been delivered before the deadline, on the deadline day and while the task has been overdue, exactly once for every recipient. Managers have been enabled to identify pending, overdue and unevenly distributed work through the dashboards and the workload screen, communication has been improved through comments, notifications and internal mail, every change has been recorded in the activity history, and reports with CSV export have been made available for planning.

From the technical point of view, it has been demonstrated through the project that a modern stack consisting of ASP.NET Core on .NET 10, Entity Framework Core with SQLite and Angular 19 could deliver a robust and maintainable application without any licence cost and without external infrastructure such as a database server, a mail server or a message broker. The layered architecture that has been enforced by automated architecture tests, the permission catalogue with Own, Team and Organisation scopes, the mandatory two-step verification, the restart safe reminder engine with database level duplicate prevention and the large suite of more than two thousand six hundred automated tests have together given confidence in the correctness and the security of the system. It has also been shown by the addition of an AI assistant that acts only after confirmation that artificial intelligence could be integrated safely into a business application of this kind.

6.2 FURTHER ENHANCEMENT

Although all the requirements of the present version of TaskFlow have been met, a number of enhancements have been identified that would make the system more powerful and more suitable for larger organisations in future. These enhancements have been identified from the known limitations that have been recorded in the technical design document, from the feedback that has been received during user acceptance testing and from the features that have commonly been found in commercial task management products. The enhancements that have been planned are listed below, and it has been expected that the modular and well-tested structure of the code would allow each of them to be added without any major change to the existing design of the system.

- Real-time push of notifications and mail counts through SignalR or WebSockets could be introduced in place of the present polling, which has been carried out on navigation and every fifteen seconds.
- File attachments on tasks, recurring tasks that would be created automatically at fixed intervals, and integration with calendar applications could be provided so that more kinds of work have been supported.
- Reminder delivery could be extended with retry and failure tracking, and the reminder intervals of a task could be made editable after the task has been created by the manager.
- Reports could be enhanced with scoping to the records that the user has been permitted to see, a department filter, a breakdown and chart by status, and streamed export for very large data sets.
- Persistent and shared rate limits, together with a server database such as SQL Server or PostgreSQL, could be introduced so that the system has been able to run on several servers for larger organisations.
- An audit log viewer for the assistant actions and for the role and permission changes, and a retention policy for the assistant conversations, could be added for the administrators by whom the system has been managed.
- The document knowledge of the assistant could be extended to PDF and Excel documents, filtering by role, uploading of documents through the user interface and streaming of answers that have been generated by the model.
- Native mobile applications for Android and iOS could be developed on top of the REST API that has already been built, so that employees could update their tasks and receive push notifications on their phones.

CHAPTER 7
BIBLIOGRAPHY AND REFERENCES

The books, the official documentation and the web resources that have been listed below have been referred to during the analysis, design, development and testing of the system and during the preparation of this report. The books have been listed in the standard bibliographic format with the name of the author, the year of publication, the title, the edition and the publisher, and the web resources have been listed with the name of the organisation or the authors, the year, the title of the page or the standard and the address from which it has been accessed. The references have been arranged separately for books and for web resources so that each of them could be located easily by the reader who has wished to study the topics of this report in greater detail.

7.1 BOOKS

[1] Pressman, R. S., & Maxim, B. R. (2019). Software Engineering: A Practitioner's Approach (9th ed.). New York: McGraw-Hill Education.

[2] Sommerville, I. (2015). Software Engineering (10th ed.). Boston: Pearson Education.

[3] Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Boston: Pearson Education.

[4] Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). New York: McGraw-Hill Education.

[5] Price, M. J. (2024). C# 13 and .NET 9 – Modern Cross-Platform Development Fundamentals (9th ed.). Birmingham: Packt Publishing.

[6] Lock, A. (2023). ASP.NET Core in Action (3rd ed.). Shelter Island, NY: Manning Publications.

[7] Smith, J. P. (2022). Entity Framework Core in Action (2nd ed.). Shelter Island, NY: Manning Publications.

[8] Freeman, A. (2022). Pro Angular: Build Powerful and Dynamic Web Apps (5th ed.). New York: Apress.

[9] Martin, R. C. (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Boston: Prentice Hall.

[10] Awad, E. M. (2010). Systems Analysis and Design (2nd ed.). New Delhi: Galgotia Publications.

7.2 WEB REFERENCES

[1] Microsoft. (2026). ASP.NET Core documentation. Retrieved from https://learn.microsoft.com/aspnet/core

[2] Microsoft. (2026). Entity Framework Core documentation. Retrieved from https://learn.microsoft.com/ef/core

[3] Microsoft. (2026). C# language documentation. Retrieved from https://learn.microsoft.com/dotnet/csharp

[4] Google. (2026). Angular documentation. Retrieved from https://angular.dev

[5] SQLite Consortium. (2026). SQLite documentation and FTS5 extension. Retrieved from https://www.sqlite.org/docs.html

[6] FluentValidation. (2026). FluentValidation documentation. Retrieved from https://docs.fluentvalidation.net

[7] Chart.js Contributors. (2026). Chart.js documentation. Retrieved from https://www.chartjs.org/docs

[8] M'Raihi, D., Machani, S., Pei, M., & Rydell, J. (2011). TOTP: Time-Based One-Time Password Algorithm (RFC 6238). Internet Engineering Task Force. Retrieved from https://www.rfc-editor.org/rfc/rfc6238

[9] Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (RFC 7519). Internet Engineering Task Force. Retrieved from https://www.rfc-editor.org/rfc/rfc7519

[10] Nottingham, M., & Wilde, E. (2016). Problem Details for HTTP APIs (RFC 7807). Internet Engineering Task Force. Retrieved from https://www.rfc-editor.org/rfc/rfc7807

[11] OWASP Foundation. (2026). OWASP Top Ten and Cheat Sheet Series. Retrieved from https://owasp.org

[12] Google. (2026). Gemini API documentation. Retrieved from https://ai.google.dev/gemini-api/docs